

Technical Handbook

Version 1.1 · June 27, 2026

Autor	Maximilian Jakob Maag B.Sc.
Audience	Administrators, Developers, Project Managers
System	Infrastructure Webshop
Stack	Go · PostgreSQL · HTMX · OpenTofu · GitLab · Linode
Scope	Architecture · Operations · Development · Product Lifecycle

Contents

I	Overview & Architecture	6
1	Introduction	7
1.1	Purpose of This Handbook	7
1.2	Target Audiences	7
1.3	Conventions Used in This Handbook	8
1.4	System At a Glance	8
2	C4 Architecture Model	9
2.1	Level 1 — System Context	9
2.1.1	Actors	9
2.1.2	External Systems	10
2.2	Level 2 — Container Diagram	11
2.3	Level 3 — Component Diagram	11
2.3.1	Component Descriptions	12
2.4	Level 4 — Key Code Flows	13
2.4.1	Order Placement Flow (Project Manager)	13
2.4.2	Order Approval & Provisioning Flow	13
2.4.3	Request–Response Lifecycle (Next.js)	14
3	Design Decisions & Technology Rationale	15
3.1	Next.js Monorepo	15
3.2	React 19 with App Router	15
3.3	Tailwind CSS v4	16
3.4	PostgreSQL with Drizzle ORM	16
3.5	Two-Container Architecture	16
3.6	OpenTofu over Terraform	17
3.7	GitLab Managed Terraform State	17
3.8	Per-Resource Isolated State	17
3.9	Webhook-Driven Pipeline Trigger	18
3.10	JWT Authentication	18
3.11	Password Hashing: bcrypt	18

II	Infrastructure as Code	20
4	GitLab Repository Concept	21
4.1	Overview	21
4.2	Repository File Structure	21
4.3	State Management	22
4.4	CI/CD Pipeline Stages	22
4.5	Webhook Integration with the Webshop	25
4.6	Dependency Between Modules	26
5	OpenTofu Modules	27
5.1	network-core	27
5.2	vm-platform	27
5.3	k8s-platform	28
5.4	shared-services	28
5.5	observability	28
III	Administrator Guide	29
6	Initial Setup & Installation	30
6.1	Prerequisites	30
6.2	Environment Variables	30
6.3	Deployment on a Docker Host	31
6.4	Deployment on Kubernetes	34
6.5	Database Migrations	40
6.6	First Login	40
7	System Configuration	41
7.1	GitLab Sources	41
7.2	Deployment Environments	41
7.3	SMTP Configuration	41
7.4	AI Translation	42
7.5	Branding	42
7.6	Currencies	42
8	User Management	43
8.1	Roles	43
8.2	Creating a Local Account	43
8.3	Editing Users	43
8.4	Deactivating Users	44
9	Audit Log	45
9.1	Logged Events	45
9.2	Filtering	45
9.3	Export	46

IV	Developer Guide	47
10	Development Environment	48
10.1	Prerequisites	48
10.2	Quick Start	48
10.3	Makefile Targets	49
11	Codebase Structure	50
11.1	Directory Layout	50
11.2	Layered Architecture	51
11.3	Template System (templ)	51
11.4	Design System	52
11.5	Adding Translations	56
11.6	Database Migrations	57
12	Testing	58
12.1	Unit Tests	58
12.2	Integration Tests (testcontainers-go)	58
12.3	End-to-End Tests with Playwright	59
13	Database Schema	62
13.1	Table Overview	63
13.2	Detailed Schema	63
13.3	Foreign Key Relationships	68
13.4	Indexes	69
14	Adding a New Feature	70
14.1	Example: Adding a New Admin Page	70
14.1.1	Step 1: Add the Table to the Drizzle Schema	70
14.1.2	Step 2: Add Shared Types	70
14.1.3	Step 3: Add the Backend Route Handler	71
14.1.4	Step 4: Add the Frontend API Wrapper	71
14.1.5	Step 5: Add the Frontend Page	71
14.1.6	Step 6: Verify	72
V	Project Manager & Product Lifecycle Guide	73
15	Ordering Infrastructure	74
15.1	The Order Lifecycle	74
15.2	Placing an Order	74
15.3	Tracking Status	74
15.4	Using an Existing Deployment as a Template	75
15.5	Decommissioning	75
16	Introducing a New Product	76
16.1	Overview	76
16.2	Step 1: Plan the Product	76

16.3	Step 2: Prepare the OpenTofu Module	77
16.4	Step 3: Configure the GitLab Webhook	77
16.5	Step 4: Create a Category (if needed)	78
16.6	Step 5: Create the Product	78
16.6.1	Basic Information	78
16.6.2	Parameters	78
16.6.3	Deployment Environments	79
16.6.4	Multiple Webhooks (Pipeline Arrays)	79
16.7	Step 6: Configure AI Translation	79
16.8	Step 7: Global and Category Parameter Sets	80
16.9	Step 8: Test the Full Order Flow	80
16.10	Step 9: Test with a Project Manager Account	81
16.11	Step 10: Product is Live	81
17	Project Management	82
17.1	Projects	82
17.2	Infrastructure Overview	82
17.3	Cost Center Assignment	82
A	Supported Languages	84
B	Environment Variable Reference	85
C	OpenTofu Module Variable Reference	87
C.1	network-core Variables	87
C.2	vm-platform Variables	87
C.3	k8s-platform Variables	88
C.4	shared-services Variables	88
C.5	observability Variables	88
D	HTTP Route Reference	90
E	Requirements Traceability Matrix	93
F	Compiling This Handbook	96

List of Figures

2.1	C4 Level 1 — System Context Diagram	9
2.2	C4 Level 2 — Container Diagram	11
2.3	C4 Level 3 — Component Diagram (Backend API)	12
4.1	Module dependency graph	26
6.1	Docker Host deployment architecture	32
6.2	Kubernetes deployment architecture	34
12.1	E2E test infrastructure	60
13.1	Simplified FK diagram. Dashed arrows = ON DELETE CASCADE; solid arrows = restricted.	68
15.1	Order lifecycle state machine	74

List of Tables

1.1	Technology stack overview	8
2.1	Component descriptions	13
4.1	GitLab repository overview	21
4.2	State name convention per module	22
6.1	Backend environment variables	31
6.2	Frontend environment variables	31
6.3	Key Helm values	36
8.1	User roles and capabilities	43
9.1	Audited actions	45
10.1	Makefile targets	49
12.1	E2E test files	60
13.1	All database tables	63
13.2	Non-primary-key indexes	69
A.1	Supported language codes	84
B.1	Backend environment variable reference	85
B.2	Frontend environment variable reference	86
C.1	network-core variables	87
C.2	vm-platform variables	88
C.3	k8s-platform variables	88
C.4	shared-services variables	88
C.5	observability variables	89
D.1	Complete REST API route reference	92
E.1	Requirements traceability matrix	95

Listings

4.1	.gitlab-ci.yml for vm-platform (webhook-triggered module)	22
4.2	.gitlab-ci.yml for singleton modules (manual apply)	24
6.1	infra/nginx/default.conf	33
6.2	Helm install with required values	35
6.3	Example secrets.values.yaml (do not commit)	35
6.4	Kubernetes Secrets for backend and frontend	37
6.5	Deployment and Service	38
6.6	Ingress with cert-manager TLS	39
6.7	HorizontalPodAutoscaler	39

Part I

Overview & Architecture

Chapter 1

Introduction

1.1 Purpose of This Handbook

Open Hybrid Cloud is an open-source, self-service portal that enables Admins and Project Managers to order, manage, and decommission IT infrastructure on Linode/Akamai Cloud through a browser-based interface. The source code is publicly available at <https://github.com/Maximilian-Maag/open-hybrid-cloud>. This handbook is the authoritative reference for everyone who operates, extends, or uses the system.

It covers:

- The complete system architecture described with the C4 model
- Infrastructure-as-Code modules and the GitLab CI/CD concept
- Architecture and design decisions with their rationale
- Step-by-step administrator operations
- Developer onboarding, code structure, and extension patterns
- The full product lifecycle from creation to decommissioning

1.2 Target Audiences

Admin	Responsible for approving orders, monitoring all infrastructure, and ordering directly without an approval step. Reads Part III and Part V .
Root	Responsible for the product catalog, system configuration, and local user accounts. Reads Part III entirely.
Developer	Extends or maintains the webshop codebase and the OpenTofu modules. Reads Part I and Part IV .
Project Manager	Places infrastructure orders and manages projects. Reads Part V .

1.3 Conventions Used in This Handbook

monospace File paths, commands, code, environment variables, and route patterns.

italic Technical terms on first use.

bold UI labels, button names, and menu items.

Note boxes Yellow-highlighted boxes providing additional context or important background information.

Important boxes Orange-highlighted boxes warning about actions that can cause data loss or service disruption.

Tip boxes Green-highlighted boxes with shortcuts and best practices.

1.4 System At a Glance

The webshop is a Next.js monorepo with a dedicated backend API and a React-based frontend. Orders are fulfilled by triggering CI/CD pipelines (GitLab, GitHub, or Bitbucket) that run OpenTofu (a Terraform-compatible open-source tool) to provision or destroy cloud resources on Linode/Akamai. Pipeline status is reported back via webhooks rather than polling.

Table 1.1: Technology stack overview

Layer	Technology	Version
Runtime	Node.js	22+
Backend	Next.js 15 (API-only)	15.x
Frontend	Next.js 15 + React	15.x / 19.x
CSS	Tailwind CSS v4	4.x
Package Mgr	pnpm (workspace)	9.x
ORM	Drizzle ORM	—
Database	PostgreSQL	15+
Auth (backend)	JWT HS256 (jose)	24 h expiry
Auth (frontend)	NextAuth v5	5.x
IaC	OpenTofu	1.9+
Cloud	Linode/Akamai	—
CI/CD	GitLab / GitHub / Bitbucket	16+ / — / —
State Backend	GitLab Managed TF State	—

Chapter 2

C4 Architecture Model

The system is described using the *C4 model* (Context, Container, Component, Code). Each level zooms in one step deeper.

2.1 Level 1 — System Context

Figure 2.1 shows who uses the webshop and which external systems it depends on.

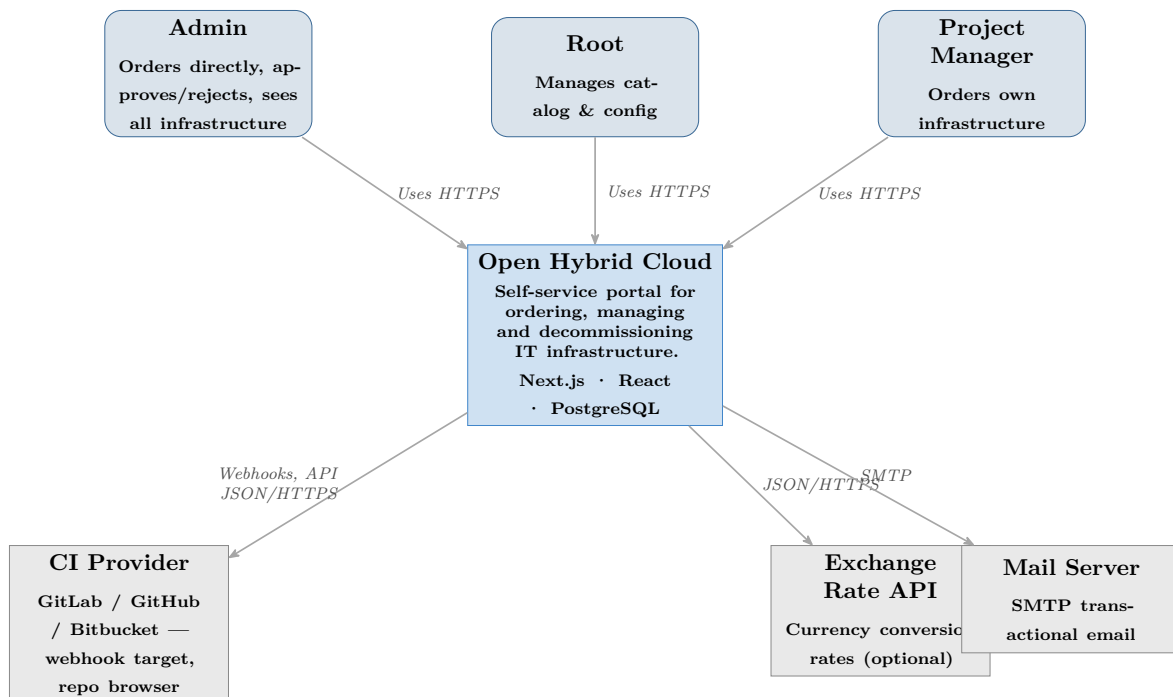


Figure 2.1: C4 Level 1 — System Context Diagram

2.1.1 Actors

Admin Authenticates via local account with JWT. Has the system role `admin`. Can order infrastructure directly (no approval required), approve or reject orders from

Project Managers, view all projects and infrastructure, and decommission any resource.

Root Uses a local account. Has the system role `root`. Manages the product catalog, system configuration (SMTP, branding, currencies), environments, CI sources, cost centers, and local users. Cannot place orders.

Project Manager Authenticates via local account with JWT. Has the system role `project_manager`. Places orders (which require Admin approval), manages their own projects, and can decommission their own infrastructure.

2.1.2 External Systems

CI Provider (GitLab / GitHub / Bitbucket) One or more self-hosted or cloud CI provider instances. The webshop connects for two purposes: (1) browsing repositories and importing `variables.tf` parameter definitions via the GitLab API, and (2) triggering provisioning and decommissioning webhooks. Pipeline status is reported back to the webshop via inbound webhook events (`POST /api/webhooks/{provider}/pipeline`).

Exchange Rate API Optional. Any HTTP JSON exchange rate endpoint. Used to refresh currency conversion rates. Configured via `EXCHANGE_RATE_API_URL` and `EXCHANGE_RATE_API_KEY`.

Mail Server Any SMTP server. Sends order confirmations, approval request notifications, approval/rejection notices, deployment completion alerts, and failure notifications. Delivered via nodemailer.

2.2 Level 2 — Container Diagram

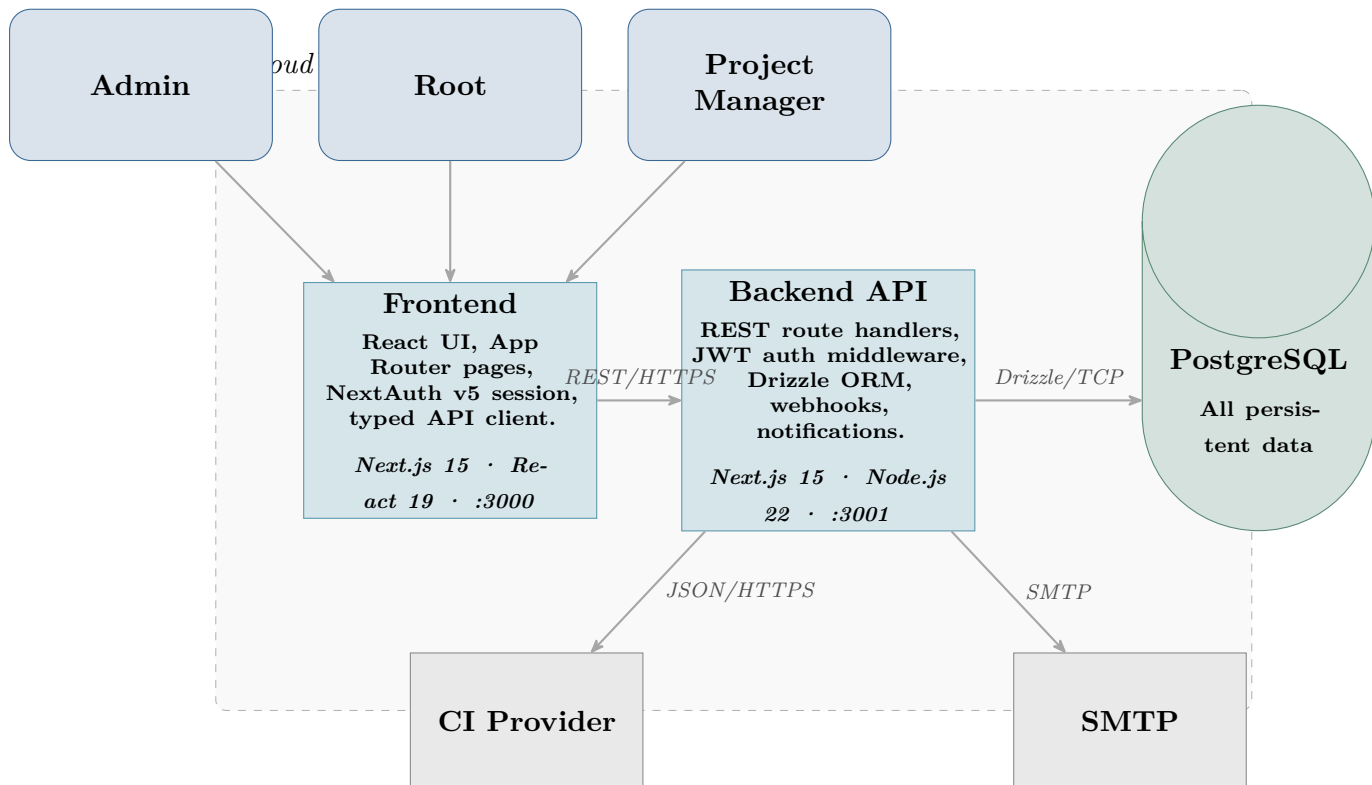


Figure 2.2: C4 Level 2 — Container Diagram

The system has three containers (plus the shared database):

Frontend (port 3000) A Next.js 15 application serving the React UI via the App Router. All pages are React Server Components or client components. It communicates with the backend exclusively via typed fetch wrappers in `src/lib/api.ts`. Authentication state is managed by NextAuth v5, which stores the JWT received from the backend in a secure server-side session.

Backend API (port 3001) A Next.js 15 application running in API-only mode — no UI pages. Route handlers under `src/app/api/` expose the full REST surface. JWT middleware (`src/lib/auth/`) verifies every authenticated request. Drizzle ORM manages all database access. The webhook handler (`src/lib/webhook/`) processes inbound pipeline status events from CI providers.

PostgreSQL Database The single source of truth. Stores the product catalog, orders, infrastructure state, audit log, branding, configuration, users, and exchange rates. The database is the only stateful component.

2.3 Level 3 — Component Diagram

Figure 2.3 decomposes the backend API container into its major logical components.

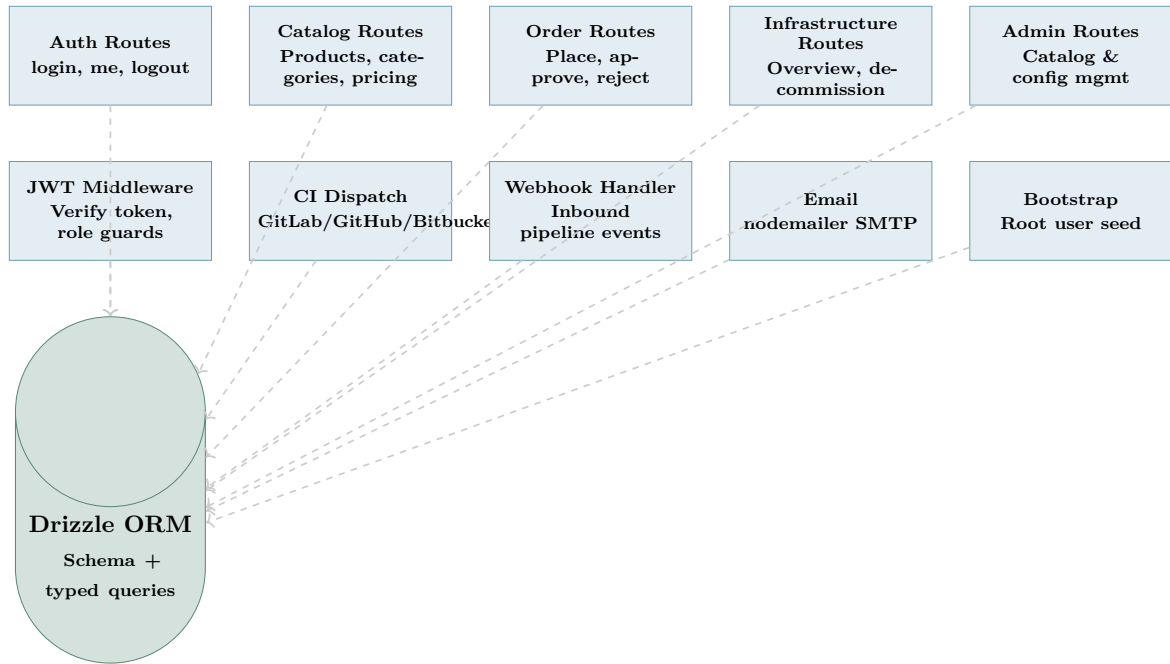


Figure 2.3: C4 Level 3 — Component Diagram (Backend API)

2.3.1 Component Descriptions

Component	Responsibility
Auth Routes	<code>POST /api/auth/login</code> validates credentials with <code>bcryptjs</code> , issues a signed HS256 JWT (24 h). <code>GET /api/auth/me</code> returns the caller's profile. <code>POST /api/auth/logout</code> is a stateless no-op (JWT expiry handles invalidation).
Catalog Routes	Returns products grouped by category (<code>GET /api/catalog</code>). Converts prices using stored exchange rates.
Order Routes	<code>POST /api/catalog/:id/order</code> creates an order and dispatches the CI webhook immediately for Admins, or parks it for approval for Project Managers. <code>POST /api/orders/:id/approve</code> and <code>POST /api/orders/:id/reject</code> complete the approval flow with email notifications and audit writes.
Infrastructure Routes	Shows deployed resources role-scoped by caller. Triggers the CI destroy webhook for decommissioning. Supports “use as template” to pre-fill a new order.
Admin Routes	All Root-only pages: categories, products (CRUD, image upload, webhook management, parameter import from <code>variables.tf</code> via GitLab API), environments, CI sources, cost centers, currencies, SMTP config, branding, users.

Component	Responsibility
JWT Middleware	Verifies Bearer tokens on every authenticated route using the <code>jose</code> library. Exports typed role guard wrappers (<code>requireAdmin</code> , <code>requireRoot</code>) used by route handlers.
CI Dispatch	Provider-agnostic module in <code>src/lib/ci/</code> that sends HTTP POST webhooks to GitLab, GitHub, or Bitbucket pipeline trigger endpoints with OpenTofu parameters as JSON.
Webhook Handler	Inbound handler at <code>POST /api/webhooks/{provider}/pipeline</code> . Maps CI pipeline status events to order/infrastructure state updates and dispatches email notifications on completion or failure.
Email	Wraps nodemailer. Silently no-ops when SMTP is not configured. Configuration is read from environment variables.
Bootstrap	On startup, seeds the initial Root account from <code>ADMIN_EMAIL</code> / <code>ADMIN_PASSWORD</code> if no users exist.
Drizzle ORM	Schema definitions and typed query builders in <code>src/lib/db/</code> . All database access goes through Drizzle; raw SQL is not used.

Table 2.1: Component descriptions

2.4 Level 4 — Key Code Flows

2.4.1 Order Placement Flow (Project Manager)

1. Frontend calls `POST /api/catalog/:id/order` with JWT in the `Authorization` header.
2. The backend route handler validates the JWT via middleware, verifies the request body, and writes an order record with status `pending_approval` to PostgreSQL via Drizzle.
3. The notification module sends an approval-request email to all Admins via nodemailer.
4. An `order_placed` row is appended to the audit log.
5. The handler returns the new order ID; the frontend navigates to the order detail page showing *Awaiting Approval*.

2.4.2 Order Approval & Provisioning Flow

1. Admin calls `POST /api/orders/:id/approve`.

2. The backend handler updates the order status to **provisioning** and reads the webhook configuration for the product-environment pair.
3. For each configured webhook (ordered by **exec_order**): an HTTP POST is sent to the CI provider trigger URL with the OpenTofu parameters as JSON.
4. The CI provider starts a pipeline and returns a pipeline ID.
5. The backend stores the pipeline ID(s) in the **pipeline_ids** JSONB column.
6. When the CI pipeline completes, the provider calls `POST /api/webhooks/{provider}/pipeline` with the result. The webhook handler updates order and infrastructure state and sends a deployment-complete email.
7. On pipeline failure, status becomes **failed** and failure emails go to the orderer and all Admins.

2.4.3 Request–Response Lifecycle (Next.js)

User interactions are handled by React Server Components or client components that call the backend REST API via typed wrappers in `src/lib/api.ts`. The backend returns JSON; the frontend re-renders the affected React subtree. NextAuth v5 maintains the user session in an encrypted server-side cookie; the session token is forwarded as a Bearer JWT on every backend API call.

Chapter 3

Design Decisions & Technology Rationale

3.1 Next.js Monorepo

Decision: The application is structured as a pnpm workspace monorepo with two Next.js applications (`apps/backend` and `apps/frontend`) and a shared types package (`packages/types`).

Rationale:

- TypeScript is shared end-to-end: the same interface definitions in `packages/types/` are consumed by both the backend route handlers and the frontend API client, making type drift a compile error rather than a runtime surprise.
- Separating the backend and frontend into distinct Next.js applications allows each to have an independent Dockerfile, independent environment variables, and an independent deploy cycle. The frontend can be updated without touching the API container.
- Next.js App Router (both apps) provides file-system-based routing, React Server Components, and built-in streaming without additional framework glue.
- pnpm workspaces deduplicate `node_modules` across packages, reducing install size and ensuring consistent package versions across the monorepo.

3.2 React 19 with App Router

Decision: The frontend is a Next.js 15 application using React 19 Server and Client Components via the App Router.

Rationale:

- **Type-safe API layer:** All backend calls are made through typed wrappers in `src/lib/api.ts`. The shared `packages/types` interfaces guarantee that the shape of every response is verified at compile time.

- **Server Components by default:** Data-fetching pages are React Server Components that call the backend on the server, avoiding client-side waterfall requests and keeping JWTs server-side.
- **Incremental interactivity:** Only the components that require event handlers or browser APIs are marked "use client", minimising the client-side JavaScript bundle.

3.3 Tailwind CSS v4

Decision: Tailwind CSS v4 is used for all styling via the single `@import "tailwindcss"` directive. No DaisyUI. No separate build step.

Rationale:

- **Zero-config build:** Tailwind v4 uses a native CSS engine that requires only the single import; there is no `tailwind.config.js` or separate `build:css` command. Next.js handles the PostCSS pass automatically.
- **Design independence:** Pure utility classes keep every style decision explicit and auditable. Brand tokens (`var(-bp)`, `var(-bs)`) drive themed surfaces without conflicting with a component library's own custom properties.

3.4 PostgreSQL with Drizzle ORM

Decision: PostgreSQL 15+, accessed via the Drizzle ORM with the `postgres` Node.js driver.

Rationale:

- **Type-safe queries:** Drizzle generates TypeScript types from the schema definition in `src/lib/db/schema.ts`. Every query result is typed; shape mismatches are caught by the TypeScript compiler.
- **Schema as code:** `drizzle-kit push` applies the schema diff directly, removing the need for manually written SQL migration files while keeping the schema version-controlled.
- **Thin abstraction:** Drizzle stays close to SQL semantics (no opaque magic), making it straightforward to write efficient queries for JSONB columns, upserts, and aggregations.
- **BYTEA columns** store product images and the branding logo directly in the database, eliminating a separate object store at this scale.

3.5 Two-Container Architecture

Decision: The backend API and the frontend UI run in separate containers (`apps/backend/Dockerfile` and `apps/frontend/Dockerfile`).

Rationale: Separating concerns at the container boundary gives each part its own deployment, scaling, and environment-variable scope. The frontend never holds database credentials; the backend never renders HTML. Nginx (or any reverse proxy) routes `/api/*` to port 3001 and all other paths to port 3000. At the anticipated load (tens of users, hundreds of orders per month), this split adds minimal operational overhead while making the security boundary explicit.

3.6 OpenTofu over Terraform

Decision: All IaC modules use OpenTofu 1.9+.

Rationale: OpenTofu is a community-governed, truly open-source fork of Terraform created after HashiCorp changed Terraform’s license to BSL 1.1 in 2023. The API, HCL syntax, provider ecosystem, and state format are fully compatible. Using OpenTofu avoids future licensing costs and complies with Open Hybrid Cloud’s open-source software policy.

3.7 GitLab Managed Terraform State

Decision: OpenTofu state is stored in GitLab’s built-in HTTP-compatible state backend (GitLab Managed Terraform State).

Rationale:

- Reuses existing GitLab infrastructure — no S3 bucket, no Terraform Cloud account, no Vault setup required.
- State is protected by GitLab’s RBAC: only project members with Developer or higher access can read or modify state.
- State locking via HTTP LOCK/UNLOCK prevents concurrent apply operations.
- State is browsable in the GitLab UI under **Infrastructure** → **Terraform states**.

3.8 Per-Resource Isolated State

Decision: Each VM and each Kubernetes cluster has its own isolated OpenTofu state (named `vm-{label}` and `k8s-{label}`).

Rationale: Shared state across multiple VMs creates blast-radius risk: a failing `apply` for one VM could lock the state and block unrelated VMs. Isolated states mean each resource’s lifecycle is entirely independent. The trade-off is a larger number of state files, which is manageable at the expected scale.

3.9 Webhook-Driven Pipeline Trigger

Decision: The webshop triggers GitLab pipelines via webhook (HTTP POST) rather than the GitLab API's pipeline-trigger endpoint.

Rationale: Webhooks are the native GitLab mechanism for external triggers. They carry a secure token for authentication and include the ability to pass variables (OpenTofu parameters) as JSON. The pipeline configuration (`.gitlab-ci.yml`) lives in the same repository as the IaC code, making the trigger configuration auditable and version-controlled.

3.10 JWT Authentication

Decision: The backend issues signed JWTs (HS256, 24-hour expiry) via the `jose` library. The frontend stores the JWT in a NextAuth v5 server-side session cookie and forwards it as a `Bearer` token on every backend API call.

Rationale:

- **Stateless backend:** No session table, no Redis — every authenticated request is self-contained. The backend verifies the JWT signature and extracts the user identity and role from the token payload without a database lookup.
- **Clean separation:** The backend exposes a pure REST interface; the frontend manages session lifecycle via NextAuth v5. The JWT is the only shared contract between the two containers.
- **Horizontal scaling:** Any backend replica can verify any token — no sticky sessions or shared session store required.
- **Logout:** Because JWTs are stateless, logout is implemented by deleting the NextAuth session cookie on the frontend. The short 24-hour expiry limits the window for a captured token. A token revocation list can be added later if needed.

Secret management: `JWT_SECRET` (backend, min. 32 chars) signs and verifies tokens. `NEXTAUTH_SECRET` (frontend) encrypts the NextAuth session cookie. Both must be kept secret and rotated on breach.

3.11 Password Hashing: bcrypt

Decision: Local user passwords are hashed with bcrypt (cost factor 12) via the `bcryptjs` npm package.

Rationale:

- bcrypt is an adaptive, memory-hard function specifically designed for password storage; it is resistant to GPU-accelerated dictionary attacks.
- Cost factor 12 takes approximately 250 ms on modern hardware — slow enough to deter brute-force attacks while still acceptable for interactive login.

- `bcryptjs` is a pure-JavaScript implementation with no native bindings, making it portable across Node.js versions and compatible with Next.js standalone output.
- Passwords are never stored in plaintext or with reversible encryption.

Alternative considered: Argon2id. More modern, but `bcrypt` is widely understood, well-tested in Node.js, and sufficient for the low login-volume use case.

Part II

Infrastructure as Code

Chapter 4

GitLab Repository Concept

4.1 Overview

The IaC layer consists of six GitLab repositories in the `open-hybrid-cloud` group. Each repository is a self-contained OpenTofu module with its own CI/CD pipeline and state. Together they provision all cloud infrastructure on Linode/Akamai.

Table 4.1: GitLab repository overview

Repository	Purpose	Trigger
<code>network-core</code>	VPC, subnets, base firewall, DNS zone	Manual
<code>vm-platform</code>	Linode VM instances (per-VM)	Webhook
<code>k8s-platform</code>	LKE Kubernetes clusters (per-cluster)	Webhook
<code>shared-services</code>	GitLab CE + Vault + Registry	Manual
<code>observability</code>	Grafana + Prometheus + Loki	Manual
<code>infra-state</code>	State backend host (no IaC code)	N/A

4.2 Repository File Structure

Every module repository (except `infra-state`) follows the same file layout:

```
<module>/
+-- .gitlab-ci.yml # CI/CD pipeline definition
+-- backend.tf     # HTTP state backend (GitLab Managed)
+-- provider.tf    # Linode provider + required versions
+-- variables.tf   # Input variable declarations
+-- main.tf        # Resource definitions
+-- outputs.tf     # Output values
```


4.3 State Management

All state files live in the `infra-state` project. The project hosts no IaC code; it is solely a container for GitLab Managed Terraform State files.

State naming convention:

Table 4.2: State name convention per module

Module	State name
network-core	network-core
vm-platform	vm-{vm_label}
k8s-platform	k8s-{cluster_label}
shared-services	shared-prod
observability	observability-prod

The backend URL pattern is:

```
https://<gitlab-host>/api/v4/projects/<INFRA_STATE_PROJECT_ID>/terraform/state/<
↳ STATE_NAME>
```

Backend configuration is passed at `tofu init` time via `-backend-config` flags, not hard-coded in `backend.tf`. This keeps credentials out of version control.

4.4 CI/CD Pipeline Stages

All module pipelines share the same three-stage structure:

1. **validate** — `tofu validate`: checks HCL syntax and provider schema compliance.
2. **plan** — `tofu plan -out=tfplan`: computes the diff and saves the plan artifact.
3. **apply** — `tofu apply tfplan`: applies the plan. For webhook-triggered modules, apply runs automatically on pipeline success. For singleton modules (`shared-services`, `observability`, `network-core`), the apply job is `when: manual` to require deliberate human confirmation.

Destroy flow: Webhook-triggered modules (`vm-platform`, `k8s-platform`) accept a `TF_ACTION=destroy` variable. When set, the apply stage runs `tofu destroy -auto-approve` instead of `tofu apply`.

Listing 4.1 shows the canonical `.gitlab-ci.yml` for the `vm-platform` module. Singleton modules differ only in the apply job: `when: manual` replaces `when: on_success`, and there is no destroy branch.

```
image: ghcr.io/opentofu/opentofu:1.9

variables:
  TF_ACTION: "apply"
```

```
TF_STATE_NAME: "vm-vm-label"
TF_VAR_linode_token: "${LINODE_TOKEN}"

.tofu_init: &tofu_init
  before_script:
    - tofu init
      -reconfigure
      -backend-config="address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}"
      -backend-config="lock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="unlock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="username=gitlab-ci-token"
      -backend-config="password=${CI_JOB_TOKEN}"
      -backend-config="lock_method=POST"
      -backend-config="unlock_method=DELETE"
      -backend-config="retry_wait_min=5"

  stages:
    - validate
    - plan
    - apply

  validate:
    stage: validate
    <<: *tofu_init
    script:
      - tofu validate
    rules:
      - if: $CI_PIPELINE_SOURCE == "trigger"

  plan:
    stage: plan
    <<: *tofu_init
    script:
      - tofu plan -out=tfplan
    artifacts:
      paths:
        - tfplan
      expire_in: 1 hour
    rules:
      - if: $CI_PIPELINE_SOURCE == "trigger"

  apply:
    stage: apply
    <<: *tofu_init
    script:
      - |
        if [ "${TF_ACTION}" = "destroy" ]; then
          tofu destroy -auto-approve
        else
          tofu apply -auto-approve tfplan
        fi
    dependencies:
      - plan
```

```

rules:
  - if: $CI_PIPELINE_SOURCE == "trigger" && $TF_ACTION == "destroy"
    when: always
  - if: $CI_PIPELINE_SOURCE == "trigger"
    when: on_success

```

Listing 4.1: .gitlab-ci.yml for vm-platform (webhook-triggered module)

Listing 4.2 shows a complete pipeline for singleton modules (`network-core`, `shared-services`, `observability`) where a human must confirm the apply step:

```

image: ghcr.io/opentofu/opentofu:1.9

variables:
  TF_STATE_NAME: "shared-prod"
  TF_VAR_linode_token: "${LINODE_TOKEN}"

.tofu_init: &tofu_init
  before_script:
    - tofu init
      -backend-config="address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}"
      -backend-config="lock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="unlock_address=${CI_API_V4_URL}/projects/${
        ↳ INFRA_STATE_PROJECT_ID}/terraform/state/${TF_STATE_NAME}/lock"
      -backend-config="username=gitlab-ci-token"
      -backend-config="password=${CI_JOB_TOKEN}"
      -backend-config="lock_method=POST"
      -backend-config="unlock_method=DELETE"
      -backend-config="retry_wait_min=5"

  stages:
    - validate
    - plan
    - apply

  validate:
    stage: validate
    <<: *tofu_init
    script:
      - tofu validate
    rules:
      - if: $CI_COMMIT_BRANCH == "main"

  plan:
    stage: plan
    <<: *tofu_init
    script:
      - tofu plan -out=tfplan
    artifacts:
      paths:
        - tfplan
      expire_in: 1 hour
    rules:
      - if: $CI_COMMIT_BRANCH == "main"

```

```
apply:
  stage: apply
  <<: *tofu_init
  script:
    - tofu apply -auto-approve tfplan
  dependencies:
    - plan
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual
```

Listing 4.2: .gitlab-ci.yml for singleton modules (manual apply)

Note

The single required CI/CD variable to set in the *infra-state* GitLab project group is `INFRA_STATE_PROJECT_ID` — the numeric project ID of the *infra-state* repository (visible in GitLab under **Settings** → **General**). `CI_JOB_TOKEN` and `CI_API_V4_URL` are provided automatically by GitLab CI. `LINODE_TOKEN` must be set with Read-/Write API access.

4.5 Webhook Integration with the Webshop

1. An Admin approves an order in the webshop (or places one directly).
2. For each webhook configured on the product-environment combination, the webshop sends:

```
POST <gitlab-webhook-url>
X-Gitlab-Token: <secret>
Content-Type: application/json

{
  "ref": "main",
  "variables": [
    {"key": "TF_VAR_vm_label", "value": "myvm-01"},
    {"key": "TF_VAR_vpc_id", "value": "12345"},
    ...
  ]
}
```

3. GitLab starts a pipeline for the `main` branch, injecting the variables as environment variables.
4. The pipeline reads `TF_VAR_*` variables, runs `tofu plan` then `tofu apply`.
5. On completion (success or failure), the webshop's polling goroutine detects the status change and updates the order record.

4.6 Dependency Between Modules

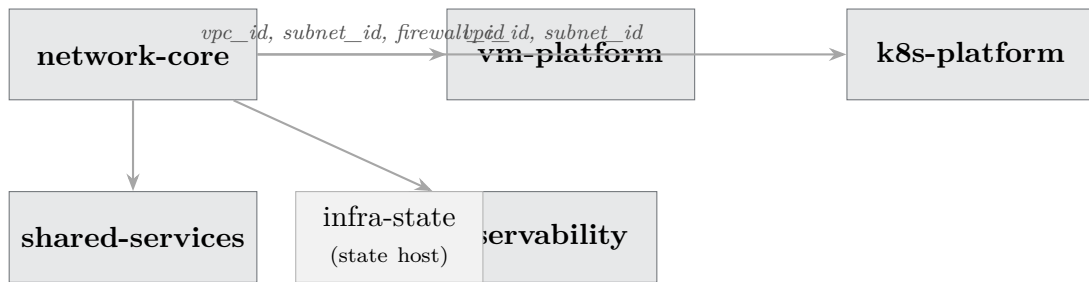


Figure 4.1: Module dependency graph

network-core must be applied first. Its outputs (*vpc_id*, *subnet_ids*, *firewall_id*) are passed as input variables to all other modules.

Chapter 5

OpenTofu Modules

5.1 network-core

Purpose: Foundational network infrastructure. Apply once per environment; no webhook trigger.

Resources created: `linode_vpc` (VPC), `linode_vpc_subnet` (subnets via `for_each`), `linode_firewall` (base inbound/outbound policy), `linode_domain` (optional DNS zone).

Key design choice: Subnets are defined as a `map(object({cidr, description}))` variable. This allows adding subnets by editing a single variable without changing the resource code.

Key outputs: `vpc_id`, `subnet_ids` (map: label $\rightarrow$ ID), `firewall_id`, `dns_zone_id`.

5.2 vm-platform

Purpose: Provision individual Linode VMs on demand. One state per VM (`vm-{label}`). Webhook-triggered by the webshop.

Resources created: `linode_firewall` (VM-specific), `linode_instance`, `linode_firewall_device` (attaches firewall to VM), `linode_volume` (optional block storage), `linode_domain_record` (optional A record).

Key design choice: The VM gets its own firewall (in addition to the base firewall from `network-core`) to allow per-VM port rules without affecting the shared base policy.

Destroy flow: Triggered by the webshop decommission action. `TF_ACTION=destroy` $\rightarrow$ `tofu destroy -auto-approve`.

5.3 k8s-platform

Purpose: Provision Linode Kubernetes Engine (LKE) clusters. One state per cluster (`k8s-{label}`). Webhook-triggered.

Resources created: `linode_firewall` (cluster-specific), `linode_lke_cluster` with dynamic `pool` blocks (supports multiple node pools with optional autoscaler), `linode_domain_record` (optional).

Key output: `kubeconfig` (base64-encoded, marked `sensitive`) — retrieve with `tofu output -raw kubeconfig | base64 -d`.

Node pool design: Node pools are a `list(object)` variable with an optional `autoscaler` sub-object, allowing any combination of fixed-size and auto-scaling pools in a single cluster.

5.4 shared-services

Purpose: Long-lived shared services: GitLab CE, HashiCorp Vault, container registry. Single state (`shared-prod`). Manual apply.

Resources created: Two `linode_instance` resources (GitLab, Vault), two `linode_volume` resources (data volumes), one `linode_firewall` with tailored inbound rules (HTTP/HTTPS/Git SSH for GitLab; Vault API 8200 restricted to RFC1918), two `linode_firewall_device` attachments, optional DNS A records.

Operational note: GitLab CE and Vault are installed via post-provisioning scripts (cloud-init or Ansible), not managed by OpenTofu. OpenTofu only provisions the infrastructure layer.

5.5 observability

Purpose: Grafana, Prometheus, and Loki on a single VM. Single state (`observability-prod`). Manual apply.

Resources created: `linode_instance`, `linode_volume` (200 GB default for metrics and logs), `linode_firewall` (Grafana port configurable, Prometheus/Loki restricted to RFC1918), optional DNS A record.

Key output: `grafana_url` — `http://{ip}:{port}`.

Part III

Administrator Guide

Chapter 6

Initial Setup & Installation

6.1 Prerequisites

- Docker 24+ (or a Kubernetes cluster)
- PostgreSQL 15+ (external or containerized)
- SMTP server credentials (optional, can be configured later)
- Microsoft Entra ID application registration for OIDC (optional for local-only operation)
- Linode API token with Read/Write scope
- GitLab instance with webhook support

6.2 Environment Variables

The system is configured via two `.env` files — one per application container. Copy the provided example files and populate the values before starting:

```
cp apps/backend/.env.example apps/backend/.env
cp apps/frontend/.env.example apps/frontend/.env
```

Backend (`apps/backend/.env`):

Table 6.1: Backend environment variables

Variable	Description	Required
<code>DATABASE_URL</code>	PostgreSQL DSN	Yes
<code>JWT_SECRET</code>	HS256 signing secret (min. 32 chars)	Yes
<code>ADMIN_EMAIL</code>	Initial Root account email	Yes
<code>ADMIN_PASSWORD</code>	Initial Root account password	Yes
<code>FRONTEND_URL</code>	Frontend origin for CORS (default: <code>http://localhost:3000</code>)	No
<code>SMTP_HOST</code>	SMTP server hostname	No
<code>SMTP_PORT</code>	SMTP port (default: 1025)	No
<code>SMTP_FROM</code>	Sender email address	No
<code>SMTP_USER</code>	SMTP authentication username	No
<code>SMTP_PASS</code>	SMTP authentication password	No
<code>SMTP_TLS</code>	<code>true</code> to enable TLS	No
<code>EXCHANGE_RATE_API_URL</code>	Exchange rate API endpoint	No
<code>ENTRA_TENANT_ID</code>	Microsoft Entra ID tenant ID (SSO)	No
<code>ENTRA_CLIENT_ID</code>	Entra ID application client ID	No
<code>ENTRA_CLIENT_SECRET</code>	Entra ID client secret	No

Frontend (`apps/frontend/.env`):

Table 6.2: Frontend environment variables

Variable	Description	Required
<code>NEXT_PUBLIC_API_URL</code>	Backend URL reachable from the browser	Yes
<code>API_URL</code>	Backend URL reachable from the SSR server	Yes
<code>NEXTAUTH_URL</code>	Canonical public URL of the frontend	Yes
<code>NEXTAUTH_SECRET</code>	NextAuth session encryption secret (min. 32 chars)	Yes

Note

SMTP can be configured (and overridden) through the Admin UI. Values saved in the database take precedence over environment variables at runtime.

6.3 Deployment on a Docker Host

This section describes a production-grade deployment on a single Linux host using Docker and an Nginx reverse proxy with TLS termination.

Architecture

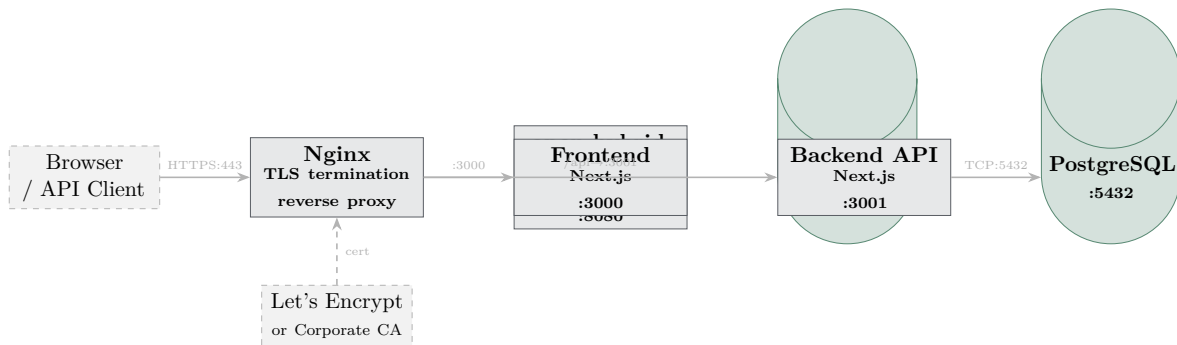


Figure 6.1: Docker Host deployment architecture

Nginx routes `/api/*` and `/api/webhooks/*` to the backend container (port 3001) and all other paths to the frontend (port 3000).

Step-by-Step Deployment Procedure

1. Install Docker and Docker Compose.

```
# Debian/Ubuntu
apt-get install -y docker.io docker-compose-v2
systemctl enable --now docker
```

2. Clone the repository.

```
git clone https://github.com/Maximilian-Maag/open-hybrid-cloud.git
cd open-hybrid-cloud
```

3. Configure environment files.

```
# Edit both files with production values
cp apps/backend/.env.example apps/backend/.env
cp apps/frontend/.env.example apps/frontend/.env
# Required backend vars: DATABASE_URL, JWT_SECRET, ADMIN_EMAIL,
  ↪ ADMIN_PASSWORD
# Required frontend vars: NEXT_PUBLIC_API_URL, API_URL, NEXTAUTH_URL,
  ↪ NEXTAUTH_SECRET
```

4. Start all containers.

```
docker compose -f infra/docker-compose.yml up -d
```

The schema is applied automatically on first startup (Drizzle push runs as part of the backend bootstrap). The root user is created from `ADMIN_EMAIL` / `ADMIN_PASSWORD` on the first `GET /api/health` call.

5. Obtain a TLS certificate.

Option A — Let's Encrypt (public domain):

```
apt-get install -y certbot python3-certbot-nginx
certbot --nginx -d shop.example.com
```

Option B — Corporate CA (internal PKI): Place the signed certificate and key at the paths referenced in [infra/docker-host/nginx.conf](#) (see below) and restart Nginx.

6. Configure Nginx as TLS-terminating reverse proxy.

Listing 6.1 shows a minimal but production-ready Nginx configuration:

```
server {
    listen 80;
    server_name shop.example.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name shop.example.com;

    ssl_certificate /etc/letsencrypt/live/shop.example.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/shop.example.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;

    client_max_body_size 10m;

    # Backend API (REST + webhooks)
    location /api/ {
        proxy_pass http://ohc-backend:3001;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }

    # Frontend (Next.js UI)
    location / {
        proxy_pass http://ohc-frontend:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 60s;
    }
}
```

Listing 6.1: `infra/nginx/default.conf`

7. Reload Nginx.

```
nginx -t && systemctl reload nginx
```

8. Verify the deployment.

Navigate to <https://shop.example.com> and log in with **ADMIN_EMAIL** / **ADMIN_PASSWORD**. Change the password immediately under **Settings** → **My Profile**.

9. (Optional) Enable automatic certificate renewal.

```
# Certbot installs a systemd timer automatically
systemctl status certbot.timer
```

Backup & Restore

```
# Backup PostgreSQL
docker exec ohc-postgres pg_dump -U postgres open_hybrid_cloud \
  | gzip > backup-$(date +%Y%m%d).sql.gz

# Restore
zcat backup-20260101.sql.gz \
  | docker exec -i ohc-postgres psql -U postgres open_hybrid_cloud
```

6.4 Deployment on Kubernetes

This section covers deploying the webshop to an existing Kubernetes cluster (e.g. the LKE cluster provisioned by `k8s-platform`).

Architecture

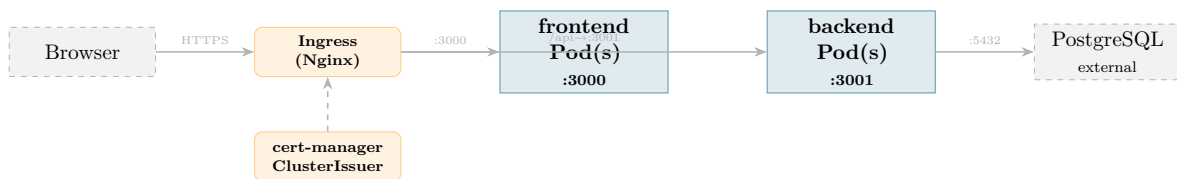


Figure 6.2: Kubernetes deployment architecture

Prerequisites

- `kubectl` and `helm` (v3.12+) configured for the target cluster
- Nginx Ingress Controller installed (`ingress-nginx`)
- `cert-manager` installed with a `ClusterIssuer` for Let's Encrypt or a corporate CA
- The images `maximilianmaag/open-hybrid-cloud-backend` and `maximilianmaag/open-hybrid-cloud-frontend` are publicly available on Docker Hub — no `imagePullSecret` required

Deployment with Helm

The repository ships a Helm chart at [infra/helm/open-hybrid-cloud/](#). It covers the full deployment in a single command and handles database migrations automatically as a pre-install/pre-upgrade hook.

```
helm upgrade --install open-hybrid-cloud \
  infra/helm/open-hybrid-cloud \
  --namespace open-hybrid-cloud \
  --create-namespace \
  --set backend.env.databaseUrl="postgres://user:pass@pg.example.com:5432/
    ↪ open_hybrid_cloud?sslmode=require" \
  --set backend.env.jwtSecret="<min-32-char-secret>" \
  --set backend.env.adminPassword="<initial-password>" \
  --set frontend.env.nextauthSecret="<min-32-char-secret>" \
  --set frontend.env.apiUrl="http://open-hybrid-cloud-backend:3001" \
  --set ingress.hosts[0].host=shop.example.com \
  --set ingress.tls[0].hosts[0]=shop.example.com \
  --set ingress.tls[0].secretName=open-hybrid-cloud-tls
```

Listing 6.2: Helm install with required values

Note

Sensitive values should be passed via a separate `secrets.values.yaml` file (not committed to version control) rather than as `-set` arguments, so they do not appear in shell history.

```
backend:
  env:
    databaseUrl: "postgres://user:pass@pg.example.com:5432/open_hybrid_cloud?
      ↪ sslmode=require"
    jwtSecret: "aabbccddeeff00112233445566778899aabbccddeeff00112233445566778899"
    adminPassword: "change-me-immediately"
frontend:
  env:
    nextauthSecret: "00112233445566778899
      ↪ aabbccddeeff00112233445566778899aabbccddeeff"
    nextauthUrl: "https://shop.example.com"
```

Listing 6.3: Example secrets.values.yaml (do not commit)

```
helm upgrade --install open-hybrid-cloud infra/helm/open-hybrid-cloud \
  --namespace open-hybrid-cloud --create-namespace \
  -f values.production.yaml \
  -f secrets.values.yaml
```

Table 6.3 lists the most important configurable values.

Table 6.3: Key Helm values

Key	Description	Default
backend.image.repository	Backend image repository	maximilianmaag/open-hybrid-cloud-backend
backend.image.tag	Backend image tag	latest
backend.replicaCount	Backend replica count	2
backend.env.databaseUrl	Backend database URL	PostgreSQL DSN (required)
backend.env.jwtSecret	Backend JWT secret	JWT signing secret (required)
backend.env.adminPassword	Backend admin password	Initial root password (required)
frontend.image.repository	Frontend image repository	maximilianmaag/open-hybrid-cloud-frontend
frontend.image.tag	Frontend image tag	latest
frontend.replicaCount	Frontend replica count	2

Step-by-Step Manual Deployment (without Helm)

1. Create the namespace.

```
kubectl create namespace open-hybrid-cloud
```

2. (Optional) Create an image pull secret.

The container image `maximilianmaag/open-hybrid-cloud` is publicly available on Docker Hub and requires no authentication. Skip this step for standard deployments. Only required if you mirror the image to a private registry:

```
kubectl create secret docker-registry my-registry-secret \
  --namespace open-hybrid-cloud \
  --docker-server=<your-registry> \
  --docker-username=<username> \
  --docker-password=<password>
```

3. Create the application Secret.

```
apiVersion: v1
kind: Secret
metadata:
  name: ohc-backend-env
  namespace: open-hybrid-cloud
type: Opaque
stringData:
  DATABASE_URL: "postgres://user:password@pg.example.com:5432/
    ↳ open_hybrid_cloud?sslmode=require"
  JWT_SECRET: "000102030405060708090
    ↳ a0b0c0d0e0f101112131415161718191a1b1c1d1e1f"
  ADMIN_EMAIL: "root@example.com"
  ADMIN_PASSWORD: "changeme"
---
apiVersion: v1
kind: Secret
metadata:
  name: ohc-frontend-env
  namespace: open-hybrid-cloud
type: Opaque
stringData:
  NEXTAUTH_SECRET: "
    ↳ f0e1d2c3b4a596870001020304050607f0e1d2c3b4a596870001020304050607"
  NEXTAUTH_URL: "https://shop.example.com"
  NEXT_PUBLIC_API_URL: "https://shop.example.com/api"
  API_URL: "http://ohc-backend:3001"
```

Listing 6.4: Kubernetes Secrets for backend and frontend

4. Apply the database schema.

The schema is pushed automatically by Drizzle on backend startup. No separate migration job is required. Verify via the health endpoint once the backend pod is running:


```
kubectl exec -n open-hybrid-cloud deploy/ohc-backend -- \
  wget -qO- http://localhost:3001/api/health
# Expected: {"status":"ok"}
```

5. Deploy the application.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
  replicas: 2
  selector:
    matchLabels:
      app: open-hybrid-cloud
  template:
    metadata:
      labels:
        app: open-hybrid-cloud
    spec:
      containers:
        - name: app
          image: maximilianmaag/open-hybrid-cloud:latest
          ports:
            - containerPort: 8080
          envFrom:
            - secretRef:
                name: open-hybrid-cloud-env
          livenessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 10
            periodSeconds: 30
          readinessProbe:
            httpGet:
              path: /health
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
          resources:
            requests:
              cpu: "100m"
              memory: "128Mi"
            limits:
              cpu: "500m"
              memory: "512Mi"
---
apiVersion: v1
kind: Service
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
```

```
selector:
  app: open-hybrid-cloud
ports:
  - port: 80
    targetPort: 8080
```

Listing 6.5: Deployment and Service

6. Configure the Ingress with TLS.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
  annotations:
    nginx.ingress.kubernetes.io/proxy-body-size: "10m"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - shop.example.com
      secretName: open-hybrid-cloud-tls
  rules:
    - host: shop.example.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: open-hybrid-cloud
                port:
                  number: 80
```

Listing 6.6: Ingress with cert-manager TLS

7. (Optional) Configure Horizontal Pod Autoscaler.

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: open-hybrid-cloud
  namespace: open-hybrid-cloud
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: open-hybrid-cloud
  minReplicas: 2
  maxReplicas: 6
  metrics:
    - type: Resource
      resource:
        name: cpu
```

```
target:
  type: Utilization
  averageUtilization: 70
```

Listing 6.7: HorizontalPodAutoscaler

8. Apply all manifests and verify.

```
kubectl apply -f secret.yaml
kubectl apply -f deployment.yaml
kubectl apply -f ingress.yaml
kubectl apply -f hpa.yaml # optional

kubectl rollout status deployment/open-hybrid-cloud -n open-hybrid-cloud
kubectl get ingress -n open-hybrid-cloud
```

Note

The polling goroutines that track GitLab pipeline status run inside each pod. With multiple replicas, advisory locks in PostgreSQL (`pg_try_advisory_lock`) ensure that only one pod polls each pipeline at a time, preventing duplicate status updates.

6.5 Database Migrations

Migrations are versioned SQL files in `internal/migrations/`. Run migrations before starting the server:

```
# Run all pending migrations
make migrate

# Or directly
go run ./cmd/migrate
```

Migrations are idempotent (IF NOT EXISTS guards). Re-running is safe.

6.6 First Login

1. Navigate to the webshop URL.
2. Click **Local Login** (not **Sign in with Microsoft**).
3. Enter the email and password from `ADMIN_EMAIL` and `ADMIN_PASSWORD`.
4. Navigate to **Settings** → **My Profile** and change the default password immediately.

Chapter 7

System Configuration

7.1 GitLab Sources

Under **Administration** → **GitLab Sources**:

1. Click **Add New Source**.
2. Enter a **Name** (label), the instance **URL**, and a **Personal Access Token** with `read_api` scope.
3. Save. The source is now available when creating environments and when browsing repositories for parameter import.

7.2 Deployment Environments

Under **Administration** → **Deployment Environments**:

1. Click **Add New Environment**.
2. Enter a **Name** (e.g. *Linode Frankfurt Production*), select the **GitLab Source**, and enter the **Webhook URL** and **Webhook Token**.
3. Save. The environment becomes available when assigning products.

Note

The Webhook URL points to a specific GitLab project pipeline trigger endpoint, e.g.:

<https://gitlab.example.com/api/v4/projects/42/ref/main/trigger/pipeline>

7.3 SMTP Configuration

Under **Administration** → **Email** (</admin/smtp>):

Enter host, port, sender address, username, password, and TLS mode. Click **Save**. The notification service reloads credentials immediately without a server restart.

7.4 AI Translation

Under **Administration** → **AI Translation** (</admin/ai-config>):

1. Select a **Provider**: Claude, OpenAI, Azure OpenAI, Ollama, or LocalAI.
2. Enter the **API endpoint** and **API key**. For Ollama/LocalAI, the endpoint is a local URL and no key is required.
3. Enter the **model name** (e.g. `claude-opus-4-7`, `gpt-4o`, `llama3.1`).
4. Click **Save**. The translator instance is replaced immediately — the next translation request uses the new provider.

7.5 Branding

Under **Administration** → **Shop Design** (</admin/branding>):

- **Shop Name** and **Subtitle** — shown in the header.
- **Primary Color** — header, footer, navigation (default: `#0f172a`).
- **Accent Color** — buttons and call-to-action elements (default: `#0ea5e9`).
- **Logo** — PNG or SVG, displayed in the header; replaces the shop name text if present.
- **Imprint** — HTML-formatted legal imprint text, shown at </impressum>.

All branding values are stored in a singleton database row and cached in-memory with a mutex for read performance.

7.6 Currencies

Under **Administration** → **Currencies**:

1. The **base currency** (default EUR) is the currency in which all product prices are stored.
2. Optionally configure an exchange rate API key and endpoint.
3. Click **Update Rates** to fetch live rates.
4. Individual rates can be overridden manually for fixed-rate environments.

Prices are displayed in the user's locale currency by looking up the browser locale and applying the stored rate.

Chapter 8

User Management

8.1 Roles

Table 8.1: User roles and capabilities

Role	Description	Auth method	DB role
Admin	Full access; direct ordering; approval	SSO	admin
Root	Catalog & config; no ordering	Local	root
Project Manager	Orders own infra; approval required	SSO	project_leader

8.2 Creating a Local Account

Under **Administration** → **Users**:

1. Click **New User**.
2. Enter **Name**, **Email**, **Password**, and select a **Role**.
3. Click **Save**.

SSO users are created automatically on first login.

8.3 Editing Users

Click **Edit** next to a user to change name, email, or role. The password can only be changed by the user themselves under **Settings** → **My Profile**.

8.4 Deactivating Users

Click **Deactivate** on the user edit page. Deactivated users cannot log in. Their orders and projects remain visible in the system.

Chapter 9

Audit Log

The audit log under `/audit` is an append-only, tamper-evident record of all system actions.

9.1 Logged Events

Table 9.1: Audited actions

Action	Trigger
order_placed	User submits an order
order_approved	Admin approves
order_rejected	Admin rejects (with comment)
order_deployed	Pipeline completes successfully
order_failed	Pipeline fails
decommission_started	User triggers decommission
decommission_complete	Destroy pipeline succeeds
user_login	Successful authentication
user_created	Admin creates a user
config_changed	System configuration updated

9.2 Filtering

Use the filter form to narrow results by:

- **Date range** (from / to)
- **User**
- **Action type**

9.3 Export

Click **Export CSV** or **Export PDF** to download the currently filtered audit log for compliance reporting.

Part IV

Developer Guide

Chapter 10

Development Environment

10.1 Prerequisites

- Go 1.23+
- Node.js 20+ (for Tailwind CSS compilation)
- Docker & Docker Compose (for local PostgreSQL)
- `golangci-lint` (static analysis — installed via `make install-requirements`)
- `pdflatex` or `lualatex` (for handbook compilation, optional)

On Ubuntu or Manjaro all prerequisites except `pdflatex` can be installed with a single command:

```
make install-requirements
```

This detects the native package manager (`pacman` or `apt-get`), installs Go, Node.js, npm, Docker, and Docker Compose, and then installs the Go tools `goimports` and `golangci-lint` via `go install`. A re-login is required afterwards for the Docker group membership to take effect.

Note

Ubuntu note: The `golang-go` package in the Ubuntu repositories may lag behind the latest Go release. If Go 1.23+ is not available, install it manually from <https://go.dev/dl/>.

10.2 Quick Start

```
# Clone and enter the repo
git clone https://github.com/Maximilian-Maag/open-hybrid-cloud.git
cd open-hybrid-cloud

# Install all required tools (Ubuntu / Manjaro)
make install-requirements
```

```
# Copy and populate environment
cp .env.example .env
# Edit .env with your local DB credentials

# Start PostgreSQL and the server in one command
make dev

# In another terminal: CSS hot-reload
make css-watch
```

`make dev` starts Docker Compose (PostgreSQL), runs CSS build, then starts the Go server with live environment variables from `.env`.

10.3 Makefile Targets

Table 10.1: Makefile targets

Target	Action
<code>make build</code>	Build CSS + Go binary
<code>make run</code>	Start server (requires running DB)
<code>make migrate</code>	Run database migrations
<code>make css</code>	Build Tailwind CSS once
<code>make css-watch</code>	Watch and rebuild CSS on change
<code>make test</code>	Run all Go tests
<code>make vet</code>	Run <code>go vet</code>
<code>make lint</code>	Run <code>golangci-lint</code>
<code>make install-requirements</code>	Install system packages + Go tools (Ubuntu/Manjaro)
<code>make docker-build</code>	Build Docker image
<code>make dev</code>	Start DB + server (development)
<code>make dev-down</code>	Stop DB + server
<code>make clean</code>	Remove build artefacts
<code>make docs</code>	Compile the LaTeX handbook to PDF
<code>make e2e</code>	Run Playwright end-to-end tests (requires Docker)

The repository also includes a local smoke test runner in `cmd/smoke/`. Run:

```
DATABASE_URL=postgres://postgres:postgres@localhost:5432/infrashop?sslmode=
↪ disable \
go run ./cmd/smoke
```

This command creates temporary rows and verifies cascading cleanup for product deletion.

Chapter 11

Codebase Structure

11.1 Directory Layout

```
open-hybrid-cloud/
+-- cmd/
|   +-- server/          # main.go -- server entry point
|   +-- migrate/         # main.go -- migration runner
|   +-- smoke/           # main.go -- smoke test runner
+-- e2e/                 # Playwright end-to-end tests (src/e2e/)
+-- docs/                # Documentation (this handbook)
|   +-- handbook.tex
|   +-- handbook.pdf     # compiled output
+-- internal/
|   +-- aitranslation/   # AI translator interface + backends
|   +-- audit/           # AuditService implementation
|   +-- auth/            # OIDC, session store, password hashing
|   +-- config/          # Config struct loaded from env
|   +-- exchange/        # Currency exchange rate service
|   +-- handler/         # HTTP handlers and route registration
|   +-- i18n/            # 25-language key-value translation map
|   +-- migrations/      # SQL migration files (001_*.sql, ...)
|   +-- model/           # Shared domain model structs
|   +-- notification/    # SMTP notification service
|   +-- repository/      # Repository interfaces
|   |   +-- postgres/    # pgx/v5 implementations
|   +-- service/         # Service interfaces
|   +-- impl/            # Service implementations
+-- ui/
|   +-- comp/            # Shared templ component library
|   |   +-- layout.templ  # Full-page layout shell + nav
|   |   +-- badge.templ  # Status badges
|   |   +-- button.templ # Button variants (Btn, BtnLink, IconBtn)
|   |   +-- card.templ   # Card, CardBody, CardHeader, PageHeader
|   |   +-- form.templ   # Field, Input, Textarea, Select, Checkbox
+-- pages/              # One package per route group
|   +-- admin/           # *.templ for all /admin/* pages
|   +-- approvals/
|   +-- audit/
|   +-- auth/
```

```
| +-- catalog/
| +-- home/
| +-- infra/
| +-- misc/
| +-- orders/
| +-- projects/
| +-- settings/
+-- static/
| +-- css/style.css      # Compiled Tailwind output
+-- ui.go                # embed.FS declarations
```

11.2 Layered Architecture

The code follows a strict layered architecture with downward-only dependencies:

Handler → Service → Repository → PostgreSQL

Handler layer ([internal/handler/](#)) HTTP handlers only. Read request, call service, render template. No business logic. No SQL.

Service layer ([internal/service/](#)) Business rules and orchestration. Calls repositories. Triggers notifications, audit writes, webhook dispatches. Defined as interfaces in [service.go](#), implemented in [impl/](#).

Repository layer ([internal/repository/](#)) Data access. Defined as interfaces (one per aggregate root). Implemented in [postgres/](#) using pgx. No business logic.

Model layer ([internal/model/](#)) Plain Go structs shared across layers. No methods beyond simple helpers.

11.3 Template System (templ)

All pages are authored as `.templ` files and compiled to Go by `templ generate ./...`. Each `.templ` file produces a corresponding `_templ.go` file which is committed alongside the source.

Page components receive a typed view struct:

```
// ui/pages/orders/orders.templ
templ OrderList(vd view.OrderListView) {
    @comp.Layout(vd.PageData) {
        // page content -- type-checked at compile time
    }
}
```

Handlers assemble the view struct and call `renderTempl`:

```
func (h *Handler) orderList(w http.ResponseWriter, r *http.Request) {
    orders, _ := h.orders.ListByUser(r.Context(), sess.UserID)
    renderTempl(w, r, orderpages.OrderList(view.OrderListView{
```

```

    PageData: h.buildPageData(w, r),
    Orders:  orders,
  )))
}

```

`buildPageData` assembles the common `view.PageData` struct providing:

- .Lang** User's active language code (e.g. "de").
- .Brand** Branding values (colours, shop name, logo flag).
- .Session** Current user session (name, role, email).
- .Flash** One-time flash message (success/error).
- .SearchQuery** Current catalog search term from the URL.

Translations are called directly in templ files:

```

{ i18n.T("orders.title", vd.Lang) } // translated string
{ string(o.Status) }                // named type requires cast

```

11.4 Design System

All UI is built from a small, self-contained component library under `src/ui/comp/`. Components are plain templ functions — no framework, no runtime dependencies.

Design Tokens

Two CSS custom properties drive every coloured surface:

var(--bp) Brand primary colour (default #0f172a, dark slate). Used for header, navigation, footer, active highlights, and price text.

var(--bs) Brand secondary / accent colour (default #0ea5e9, sky blue). Used for CTA buttons, active language badges, and focus rings.

The values are injected by `comp.Layout` as inline CSS custom properties on `<html>`. Every component should reference these variables rather than hardcoding colours:

```

<!-- correct -->
<div style="background-color:var(--bp)">...</div>
<a style="color:var(--bs)">...</a>

<!-- wrong: breaks custom branding -->
<div class="bg-[#0f172a]">...</div>

```

Layout Shell (`comp.Layout`)

Every page is wrapped in `comp.Layout`:

```
templ MyPage(vd view.MyView) {  
    @comp.Layout(vd.PageData) {  
        // page content here  
    }  
}
```

`comp.Layout` renders: `<head>` with meta tags and stylesheets, sticky header with brand logo / search bar / user menu / language picker, the top navigation bar with role-based links, a flash banner slot, the `<main>` content area (2XL max-width, horizontal padding), and the footer. Nothing else should set `<html>`, `<head>`, or `<body>`.

Page Structure

`comp.PageHeader(title, backHref)` Full-width page title row with an optional back arrow link. Use at the top of every dedicated page.

`comp.PageHeaderAction(title, backHref, action)` Same as above but with an action slot on the right (e.g. a primary button).

`comp.Section(title)` A titled content group with a subtle separator. Use to divide a page into logical parts.

`comp.EmptyState(message)` Centred placeholder message for empty lists or zero results.

```
@comp.PageHeaderAction("Bestellungen", "/", action) {  
    @comp.BtnLink("Neue Bestellung", "/catalog",  
        comp.VPrimary, comp.SzMD)  
}  
@comp.Section("Offene Bestellungen") {  
    if len(orders) == 0 {  
        @comp.EmptyState("Keine Bestellungen vorhanden.")  
    }  
    // list rows...  
}
```

Cards

`comp.Card()` White surface with border and subtle shadow. Use as a generic content container.

`comp.CardBody()` Adds consistent padding inside a `Card`.

`comp.CardHeader(title)` A titled header strip for a card.

```
@comp.Card() {  
    @comp.CardHeader("Details") {}  
    @comp.CardBody() {  
        // form fields, table, prose, etc.  
    }  
}
```


Buttons

- comp.Btn(label, variant, size, templ.Attributes)** A `<button>` element. Pass extra HTML attributes (e.g. `hx-post`, `disabled`) via the fourth argument.
- comp.BtnLink(label, href, variant, size)** An `<a>` element styled as a button. Use for navigation CTAs.
- comp.IconBtn(variant, size, templ.Attributes)** Square icon button; children supply the icon SVG.
- comp.BtnClass(variant, size)** Returns just the Tailwind class string. Use when you need to apply button styling to an element not covered by the components above (e.g. an HTMX-powered `<button>` inside a `templ.children...` slot).

Variants:

- | | |
|------------------------|--|
| comp.VPrimary | Filled, uses <code>var(-bp)</code> background — main CTAs. |
| comp.VSecondary | Outlined border button — secondary actions. |
| comp.VGhost | Transparent, text-only — low-emphasis actions. |
| comp.VDanger | Red filled — destructive actions (delete, reject). |

Sizes:

- | | |
|------------------|---|
| comp.SzXS | Extra small — inline or table-row actions. |
| comp.SzSM | Small — secondary actions, compact layouts. |
| comp.SzMD | Medium (default) — primary page-level CTAs. |

```
// Primary CTA
@comp.BtnLink("Jetzt bestellen", "/catalog/"+id,
              comp.VPrimary, comp.SzMD)

// Danger action with HTMX
@comp.Btn("Ablehnen", comp.VDanger, comp.SzSM,
          templ.Attributes{"hx-delete": "/orders/"+id,
                           "hx-confirm": "Wirklich ablehnen?"})

// Icon button (custom SVG inside)
@comp.IconBtn(comp.VGhost, comp.SzXS,
              templ.Attributes{"aria-label": "Bearbeiten"}) {
  <svg .../>
}
```

Badges

- comp.Badge(label, color)** A small inline pill. Use `comp.BadgeGreen`, `comp.BadgeRed`, `comp.BadgeYellow`, `comp.BadgeBlue`, or `comp.BadgeGray` for the color argument.
- comp.StatusBadge(status)** Convenience wrapper that maps an order/resource status string to the appropriate colour via `comp.StatusBadgeColor`. No need to branch on status in the template.

```
// Generic badge
@comp.Badge("Aktiv", comp.BadgeGreen)
```

```
@comp.Badge("Fehler", comp.BadgeRed)

// Status-aware badge (recommended for orders)
@comp.StatusBadge(string(order.Status))
```

Status colour mapping (via `comp.StatusBadgeColor`):

active, approved, completed	Green
pending	Yellow
failed, rejected	Red
provisioning, decommissioning	Blue
Everything else	Gray

Forms

All form fields use the components in `ui/comp/form.templ` for consistent label alignment, spacing, and required-marker rendering.

`comp.Field(label, name, required)` A `<label>` + slot wrapper. Place an `Input`, `Select`, or `Textarea` inside it.

`comp.Input(name, type, placeholder, value, attrs)` Styled `<input>`. Pass extra attributes (e.g. `hx-get`, `min`, `step`) via the last argument.

`comp.Textarea(name, placeholder, value, rows, attrs)` Styled `<textarea>`.

`comp.Select(name, attrs)` Styled `<select>` wrapper; children are `<option>` elements written inline.

`comp.Checkbox(name, label, checked)` A labelled checkbox with correct `for/id` wiring.

```
<form method="POST" action="/admin/products">
  @comp.Field("Produktname", "name", true) {
    @comp.Input("name", "text", "z.B. VM-Plattform",
               product.Name, nil)
  }
  @comp.Field("Beschreibung", "description", false) {
    @comp.Textarea("description", "", product.Description,
                  4, nil)
  }
  @comp.Field("Kategorie", "category_id", true) {
    @comp.Select("category_id", nil) {
      for _, c := range categories {
        <option value={ fmt.Sprint(c.ID) }
          selected?={ c.ID == product.CategoryID }>
          { c.Name }
        </option>
      }
    }
  }
  @comp.Field("Aktiv", "active", false) {
    @comp.Checkbox("active", "Produkt ist sichtbar",
                  product.Active)
```

```

    }
    @comp.Btn("Speichern", comp.VPrimary, comp.SzMD, nil)
</form>

```

Flash Messages

Flash messages are set server-side via `redirectWithFlash` and rendered automatically by `comp.Layout` from `pd.Flash`. HTMX responses that want to show a flash message use the `HX-Trigger` response header instead:

```

// In a handler that returns an HTMX partial:
w.Header().Set("HX-Trigger", `{"showFlash":{"kind":"success",
    "message":"Gespeichert"}}`)

```

Component Conventions

- **Never hardcode colours.** Always use `var(-bp)`, `var(-bs)`, or Tailwind's neutral palette (`slate-*`). Hardcoded hex values break custom branding.
- **Use `comp.VPrimary` for the single most important action** on a page. Use `comp.VSecondary` or `comp.VGhost` for everything else.
- **No JavaScript.** Interactive disclosure (menus, modals, accordions) uses native `<details>/<summary>`. Server-side partial updates use HTMX attributes. Never add `<script>` tags to component files.
- **No inline style except for brand tokens.** Arbitrary colours, sizes, and spacing belong in Tailwind utility classes, not `style=""` attributes.
- **Prefer `comp.StatusBadge` over manual branching.** The mapping is maintained centrally; adding a new status only requires updating `comp.StatusBadgeColor`.

11.5 Adding Translations

All UI strings live in `internal/i18n/i18n.go` as a `map[string]map[string]string`. To add a new key:

```

// In translations map:
"my.new.key": {
    "en": "English text",
    "de": "Deutscher Text",
    "fr": "Texte francais",
    // ... all 25 language codes
},

```

Use in templates with `{{t "my.new.key" .Lang}}`.

11.6 Database Migrations

Migration files follow the naming pattern `NNN_description.sql` and are executed in alphabetical order. Write migrations defensively:

```
ALTER TABLE products
  ADD COLUMN IF NOT EXISTS image_mime TEXT NOT NULL DEFAULT 'image/jpeg';

CREATE TABLE IF NOT EXISTS app_config (
  id INT PRIMARY KEY DEFAULT 1 CHECK (id = 1),
  smtp_host TEXT NOT NULL DEFAULT ''
  -- ...
);
```

Important

Never modify a migration that has already been applied to production. Always create a new migration file for schema changes.

Chapter 12

Testing

The project uses three complementary test layers. All three are invoked automatically in CI; locally the most common commands are `make test` (fast, no Docker needed) and `make e2e` (full browser stack, requires Docker).

12.1 Unit Tests

Unit tests live next to the code they test (`*_test.go`) and use Go's standard `testing` package. They cover:

- **Service layer** (`internal/service/impl/`) — all service methods are tested with in-memory stubs that implement the repository interfaces. No database, no network.
- **Handler helpers** (`internal/handler/`) — language detection (`parseAcceptLanguage`, `detectLang`) and the login rate-limiter are tested with direct function calls.
- **Polling worker** (`internal/polling/`) — `extractProjectID` is tested with table-driven cases.

Run unit tests:

```
make test      # runs go test ./...
make vet       # go vet (zero warnings required)
make lint     # golangci-lint
```

12.2 Integration Tests (testcontainers-go)

Repository-layer tests in `internal/repository/postgres/` spin up a real **PostgreSQL 16 Alpine** container via `testcontainers-go` and run all migrations before the first test.

TestMain

```
// testmain_test.go
func TestMain(m *testing.M) {
    ctr, err := tcpostgres.Run(ctx, "postgres:16-alpine",
        tcpostgres.WithDatabase("testdb"),
        testcontainers.WithWaitStrategy(
            wait.ForLog("database system is ready to accept connections").
                WithOccurrence(2).WithStartupTimeout(60*time.Second),
        ),
    )
    // runs migrations, opens pgxpool
    os.Exit(m.Run())
}
```

Test isolation

Every test that writes data calls `resetDB(t)` first, which truncates all data tables and restarts identity sequences:

```
func resetDB(t *testing.T) {
    _, err := testPool.Exec(ctx,
        "TRUNCATE audit_log, orders, products, ... RESTART IDENTITY CASCADE")
}
```

Fixture helper

Tests that need FK-dependent data call `newFixture(t)`, which creates the full dependency chain in one call:

```
gitlab_source -> deployment_environment -> user
               -> category -> product -> project
```

Integration tests require Docker; if the container cannot start the test binary exits with code 0 (skip, not fail) so CI stays green on environments without Docker.

12.3 End-to-End Tests with Playwright

End-to-end tests live in [src/e2e/](#) and drive a real **Chromium** browser via [playwright-go](#). The full application stack — PostgreSQL, all migrations, all service wiring — is started in-process for each test run.

Architecture

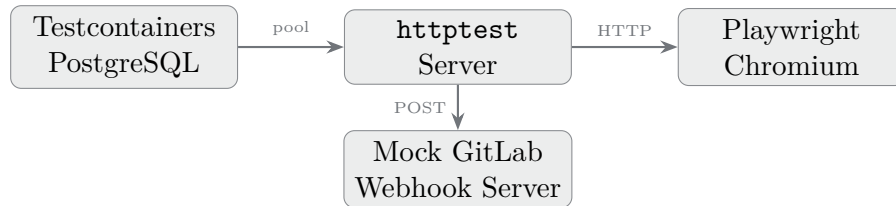


Figure 12.1: E2E test infrastructure

Testcontainers PostgreSQL A real `postgres:16-alpine` container is started by `TestMain`. All 12 migrations run against it and the admin user is bootstrapped (email `admin@e2etest.local`, password `e2eAdmin123!`).

`app.New()` The server wiring function from [internal/app/wire.go](#) assembles the full dependency graph (repos, services, handlers) and returns an `http.Handler`.

`httptest.NewServer` Wraps the handler in a local HTTP server on a random port. Playwright's `BaseURL` is set to this address.

Mock GitLab webhook server Approving an order triggers a GitLab pipeline via HTTP POST. A second `httptest.Server` intercepts these calls and returns `{"id":42}` so the approval flow completes without a real GitLab instance.

Chromium (headless) One browser is launched for the entire test run. Each test gets its own *browser context* (isolated cookie jar) via `newPage(t)`.

Test file overview

Table 12.1: E2E test files

File	Tests	What is verified
auth_test.go	3	Login success, invalid credentials, logout dropdown
approval_test.go	3	Empty list, Approve button, Reject button
catalog_test.go	3	Page loads, empty state, product navigation

Shared helpers

`newPage(t)` Creates a fresh browser context and page; the context is closed in `t.Cleanup`. Each test gets an empty cookie jar, so sessions never leak between tests.

`loginAs(t, page, email, password)` Navigates to `/login`, fills both fields, clicks `Submit`, and waits for the redirect to `/`.

`resetData(t)` Truncates all data tables (except `users`) with `RESTART IDENTITY CASCADE`, so the bootstrapped admin user survives across tests.

seedPendingOrder(t) Inserts the full FK chain (gitlab_source, environment, category, product, project, order) with **status = 'pending_approval'**. Uses **webhookBaseUrl** so the mock server intercepts the approval webhook call.

Key selector conventions

Because approvals use plain HTML `<form>` elements (no HTMX), tests use stable **action-attribute selectors**:

```
// approve button for a specific order
approveForm := page.Locator(
    fmt.Sprintf('form[action="/approvals/%d/approve"]', fix.orderID))
approveForm.Locator('button[type="submit"]').Click()

// reject form is inside a <details> -- open summary first
page.Locator('details.mt-4 > summary').First().Click()
noteInput := page.Locator('input[name="note"]')
noteInput.WaitFor()           // waits until visible
noteInput.Fill("reason")
```

After a form submit the 303 redirect reloads the same URL. `WaitForLoadState(NetworkIdle)` is used instead of `WaitForURL` (which would return immediately because the URL does not change).

Running the tests

```
# Headless (CI / default)
make e2e

# Headed -- watch the browser while tests run
E2E_HEADED=true make e2e

# First run downloads Chromium (~170 MB) into ~/.cache/ms-playwright/
```

If Docker is unavailable the test binary exits with code 0 (skip, not fail), consistent with the integration test policy.

Adding a new E2E test

1. Call `resetData(t)` at the start if the test writes data.
2. Use `newPage(t)` to get an isolated page.
3. Use `loginAs` to authenticate if needed.
4. Prefer `form[action="..."]` and `input[name="..."]` selectors over CSS classes (classes change; actions rarely do).
5. After a form submit call `page.WaitForLoadState(NetworkIdle)` before asserting.

Chapter 13

Database Schema

The schema is managed by Drizzle ORM. The authoritative definition is in [apps/backend/src/lib/db/schema.ts](#); `make db-push` (`drizzle-kit push`) applies any schema changes to the database. The table below shows the current effective schema.

13.1 Table Overview

Table 13.1: All database tables

Table	Purpose
<code>users</code>	User accounts (local + SSO) with role-based access
<code>categories</code>	Product catalogue categories
<code>products</code>	Infrastructure products offered in the catalogue
<code>product_translations</code>	Multi-language name + description per product
<code>parameters</code>	Configurable inputs for global/category/product scope
<code>ci_sources</code>	CI provider credentials (GitLab / GitHub / Bitbucket)
<code>deployment_environments</code>	Named environments linked to a CI source
<code>product_environments</code>	Junction: which products are available in which environment (with price)
<code>product_webhooks</code>	Optional per-product pipeline trigger overrides
<code>cost_centers</code>	Billing cost centres referenced by projects and orders
<code>projects</code>	User-owned projects that group orders
<code>orders</code>	Individual infrastructure provisioning requests
<code>infrastructure_elements</code>	Deployed instances tracked per order
<code>audit_log</code>	Append-only log of significant user actions
<code>exchange_rates</code>	Cached currency conversion rates
<code>branding</code>	Single-row branding config (logo, colours, texts)
<code>app_config</code>	Single-row application config (SMTP, AI)

13.2 Detailed Schema

Each table is described with its columns (post all migrations), primary key, foreign keys, and notable constraints.

users

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>email</code>	TEXT	—	NOT NULL, UNIQUE
<code>name</code>	TEXT	—	NOT NULL
<code>role</code>	TEXT	—	CHECK: <code>admin</code> , <code>project_manager</code> , <code>root</code>
<code>sso_sub</code>	TEXT	NULL	UNIQUE; set for SSO accounts
<code>password_hash</code>	TEXT	NULL	set for local accounts
<code>active</code>	BOOLEAN	TRUE	soft-disable without deletion
<code>created_at</code>	TIMESTAMPTZ	NOW()	

Constraint: `sso_sub` IS NOT NULL OR `password_hash` IS NOT NULL (every account must have at least one credential).

categories

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>name</code>	TEXT	—	NOT NULL
<code>display_order</code>	INT	0	controls sidebar sort order

products

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>category_id</code>	BIGINT	—	NOT NULL, FK → <code>categories(id)</code> ON DELETE CASCADE
<code>base_language</code>	TEXT	'de'	fallback language for translations
<code>image</code>	BYTEA	NULL	product image stored inline
<code>created_at</code>	TIMESTAMPTZ	NOW()	

product_translations

Column	Type	Default	Notes
<code>product_id</code>	BIGINT	—	PK, FK → <code>products(id)</code> ON DELETE CASCADE
<code>language_code</code>	TEXT	—	PK, e.g. <code>de</code> , <code>en</code> , <code>fr</code>
<code>name</code>	TEXT	—	NOT NULL
<code>description</code>	TEXT	"	NOT NULL

parameters

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>scope</code>	TEXT	—	CHECK: <code>global</code> , <code>category</code> , <code>product</code>
<code>scope_id</code>	BIGINT	0	ID of the scoped entity (0 for global)
<code>environment_id</code>	BIGINT	0	0 = applies to all environments
<code>name</code>	TEXT	—	NOT NULL; variable name passed to pipeline
<code>type</code>	TEXT	—	CHECK: <code>string</code> , <code>number</code> , <code>bool</code> , <code>dropdown</code>
<code>description</code>	TEXT	"	
<code>default_value</code>	TEXT	"	
<code>required</code>	BOOLEAN	FALSE	
<code>sensitive</code>	BOOLEAN	FALSE	renders as <code>type="password"</code> in UI

ci_sources

Formerly `gitlab_sources`. Supports multiple CI providers via the `provider` discriminator column.

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>name</code>	TEXT	—	NOT NULL; human-readable label
<code>provider</code>	TEXT	'gitlab'	CHECK: <code>gitlab</code> , <code>github</code> , <code>bitbucket</code>
<code>url</code>	TEXT	—	NOT NULL; CI provider base URL
<code>access_token</code>	TEXT	—	NOT NULL

deployment_environments

Column	Type	Default	Notes
<code>id</code>	BIGSERIAL	—	PK
<code>name</code>	TEXT	—	NOT NULL
<code>description</code>	TEXT	"	
<code>ci_source_id</code>	BIGINT	—	NOT NULL, FK → <code>ci_sources(id)</code>
<code>webhook_url</code>	TEXT	—	NOT NULL; pipeline trigger URL
<code>webhook_token</code>	TEXT	—	NOT NULL; pipeline trigger token

product_environments

Junction table linking products to environments with pricing.

Column	Type	Default	Notes
<code>product_id</code>	BIGINT	—	PK, FK → <code>products(id)</code> CASCADE
<code>environment_id</code>	BIGINT	—	PK, FK → <code>deployment_environments(id)</code>
<code>price</code>	NUMERIC(12,2)	0	monthly price
<code>currency</code>	TEXT	'EUR'	ISO 4217 code
<code>cost_center_mode</code>	TEXT	'project'	CHECK: <code>project</code> , <code>select</code> , <code>overhead</code>
<code>forced_cost_center</code>	BOOLEAN	FALSE	locks cost centre selection in order form

product_webhooks

Optional per-product pipeline overrides; if rows exist for a product+environment pair they replace the environment's default `webhook_url`. Multiple rows are fired in ascending `exec_order`.

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
product_id	BIGINT	—	NOT NULL, FK → products(id) CASCADE
environment_id	BIGINT	—	NOT NULL, FK → deployment_environments(id)
name	TEXT	—	NOT NULL; descriptive label
webhook_url	TEXT	—	NOT NULL
webhook_token	TEXT	—	NOT NULL
exec_order	INT	0	ascending; ties fire concurrently

cost_centers

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
code	TEXT	—	NOT NULL, UNIQUE; billing code
name	TEXT	—	NOT NULL
active	BOOLEAN	TRUE	inactive centres are hidden in UI

projects

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
name	TEXT	—	NOT NULL
description	TEXT	"	
owner_id	BIGINT	—	NOT NULL, FK → users(id)
cost_center_id	BIGINT	NULL	FK → cost_centers(id)
created_at	TIMESTAMPTZ	NOW()	

orders

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
project_id	BIGINT	—	NOT NULL, FK → projects(id) CASCADE
product_id	BIGINT	—	NOT NULL, FK → products(id) CASCADE
environment_id	BIGINT	—	NOT NULL, FK → deployment_environments(id)
user_id	BIGINT	—	NOT NULL, FK → users(id)
status	TEXT	'pending_approval'	see lifecycle below
parameters	JSONB	'{'	key/value map; passed to pipeline
cost_center_id	BIGINT	NULL	FK → cost_centers(id)
rejection_note	TEXT	"	filled on rejection
pipeline_id	JSONB	'[]'	array of GitLab pipeline IDs
created_at	TIMESTAMPTZ	NOW()	
updated_at	TIMESTAMPTZ	NOW()	

Status lifecycle: pending_approval → approved / rejected → provisioning → completed / failed → decommissioning → decommissioned.

infrastructure_elements

One row per deployed pipeline; created when an order is approved.

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
order_id	BIGINT	—	NOT NULL, FK → orders(id)
project_id	BIGINT	—	NOT NULL, FK → projects(id) CASCADE
environment_id	BIGINT	—	NOT NULL, FK → deployment_environments(id)
product_id	BIGINT	—	NOT NULL, FK → products(id) CASCADE
status	TEXT	'provisioning'	mirrors order status values
parameters	JSONB	'{}'	copy of order parameters at deploy time
pipeline_id	JSONB	'[]'	tracked pipeline IDs
outputs	JSONB	'{}'	key/value map of OpenTofu outputs
deployed_at	TIMESTAMPTZ	NOW()	

audit_log

Append-only; rows are never updated or deleted.

Column	Type	Default	Notes
id	BIGSERIAL	—	PK
user_id	BIGINT	—	NOT NULL, FK → users(id)
action	TEXT	—	NOT NULL; e.g. order.created
entity_id	BIGINT	—	NOT NULL; PK of the referenced entity
details	TEXT	"	extra context (e.g. rejection note)
created_at	TIMESTAMPTZ	NOW()	

exchange_rates

Column	Type	Default	Notes
currency_code	TEXT	—	PK; ISO 4217, e.g. EUR
rate	NUMERIC(18,6)	—	NOT NULL; relative to base currency (EUR = 1.0)
updated_at	TIMESTAMPTZ	NOW()	refreshed via POST /admin/currencies/refresh

branding

Singleton row (id = 1). Written via [POST /admin/branding](#).

Column	Type	Default	Notes
id	INT	1	PK, CHECK (id = 1)
logo_data	BYTEA	NULL	uploaded logo image
logo_mime	TEXT	'image/png'	MIME type of the logo
primary_color	TEXT	'#0f172a'	var(-bp) CSS token
secondary_color	TEXT	'#0ea5e9'	var(-bs) CSS token
shop_name	TEXT	"	overrides APP_NAME env var
shop_subtitle	TEXT	"	overrides APP_SUBTITLE env var
imprint_text	TEXT	"	rendered on /impressum

app_config

Singleton row (id = 1). Written via [POST /admin/smtp](#) and [POST /admin/ai-config](#). DB values override environment variables at runtime.

Column	Type	Default	Notes
id	INT	1	PK, CHECK (id = 1)
smtp_host	TEXT	"	SMTP server hostname
smtp_port	TEXT	'587'	
smtp_from	TEXT	"	sender address
smtp_username	TEXT	"	
smtp_password	TEXT	"	stored in plain text
smtp_tls	BOOLEAN	FALSE	enable STARTTLS
ai_provider	TEXT	"	e.g. claude, openai
ai_endpoint	TEXT	"	custom API base URL
ai_api_key	TEXT	"	stored in plain text
ai_model	TEXT	"	model ID string

13.3 Foreign Key Relationships

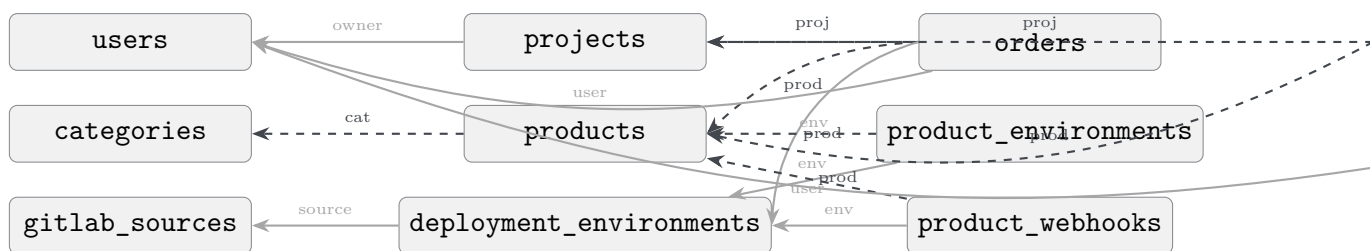


Figure 13.1: Simplified FK diagram. Dashed arrows = ON DELETE CASCADE; solid arrows = restricted.

Note

The cascade policy ensures that deleting a **product**, **project**, or **category** automatically removes all dependent rows (orders, infra elements, translations, etc.) without requiring explicit cleanup in application code.

13.4 Indexes

Table 13.2: Non-primary-key indexes

Index name	Table	Columns
idx_products_category	products	category_id
idx_orders_user	orders	user_id
idx_orders_status	orders	status
idx_orders_project	orders	project_id
idx_infra_project	infrastructure_elements	project_id
idx_infra_status	infrastructure_elements	status
idx_projects_owner	projects	owner_id
idx_audit_user	audit_log	user_id
idx_audit_action	audit_log	action
idx_parameters_scope	parameters	scope, scope_id, environment_id
idx_product_webhooks	product_webhooks	product_id, environment_id, exec_order

Chapter 14

Adding a New Feature

This chapter describes the canonical pattern for adding a feature end-to-end in the Next.js monorepo.

14.1 Example: Adding a New Admin Page

Goal: Add an admin endpoint that lists and creates widgets stored in a new `widgets` database table, and a corresponding admin UI page.

14.1.1 Step 1: Add the Table to the Drizzle Schema

In `apps/backend/src/lib/db/schema.ts`, add the table definition:

```
export const widgets = pgTable('widgets', {
  id:      bigserial('id', { mode: 'number' }).primaryKey(),
  name:    text('name').notNull(),
  active:  boolean('active').notNull().default(true),
  createdAt: timestamp('created_at', { withTimezone: true })
    .notNull().defaultNow(),
})
```

Apply the schema change:

```
make db-push
```

14.1.2 Step 2: Add Shared Types

In `packages/types/src/index.ts`, add the interface:

```
export interface Widget {
  id:      number
  name:    string
  active:  boolean
  createdAt: string
}
```

14.1.3 Step 3: Add the Backend Route Handler

Create `apps/backend/src/app/api/admin/widgets/route.ts`:

```
import { NextRequest, NextResponse } from 'next/server'
import { requireRole, isAuth } from '@lib/auth/middleware'
import { db } from '@lib/db/client'
import { widgets } from '@lib/db/schema'

export async function GET(req: NextRequest) {
  const session = await requireRole('root')(req)
  if (!isAuth(session)) return session

  const rows = await db.select().from(widgets).orderBy(widgets.name)
  return NextResponse.json(rows)
}

export async function POST(req: NextRequest) {
  const session = await requireRole('root')(req)
  if (!isAuth(session)) return session

  const { name } = await req.json()
  if (!name) return NextResponse.json(
    { error: 'name required' }, { status: 400 })

  const [row] = await db.insert(widgets).values({ name }).returning()
  return NextResponse.json(row, { status: 201 })
}
```

14.1.4 Step 4: Add the Frontend API Wrapper

In `apps/frontend/src/lib/api.ts`, the generic `get` and `post` wrappers already exist. Call them from any Server Component or client action:

```
import { get, post } from '@lib/api'
import type { Widget } from '@open-hybrid-cloud/types'

export const fetchWidgets = (token: string) =>
  get<Widget[]>('/api/admin/widgets', token)

export const createWidget = (name: string, token: string) =>
  post<Widget>('/api/admin/widgets', { name }, token)
```

14.1.5 Step 5: Add the Frontend Page

Create `apps/frontend/src/app/admin/widgets/page.tsx`:

```
import { auth } from '@lib/auth'
import { fetchWidgets } from '@lib/api/widgets'
import { redirect } from 'next/navigation'

export default async function AdminWidgetsPage() {
  const session = await auth()
  if (!session) redirect('/login')
```

```
const widgets = await fetchWidgets(session.apiToken ?? '')

return (
  <div className="p-6">
    <h1 className="text-2xl font-bold mb-4">Widgets</h1>
    <ul>
      {widgets.map(w => (
        <li key={w.id} className="py-2 border-b">{w.name}</li>
      ))}
    </ul>
  </div>
)
```

14.1.6 Step 6: Verify

```
make type-check # TypeScript must pass clean
make lint       # ESLint must pass clean
```

Navigate to <http://localhost:3000/admin/widgets> and confirm the page renders correctly.

Part V

Project Manager & Product Lifecycle Guide

Chapter 15

Ordering Infrastructure

15.1 The Order Lifecycle

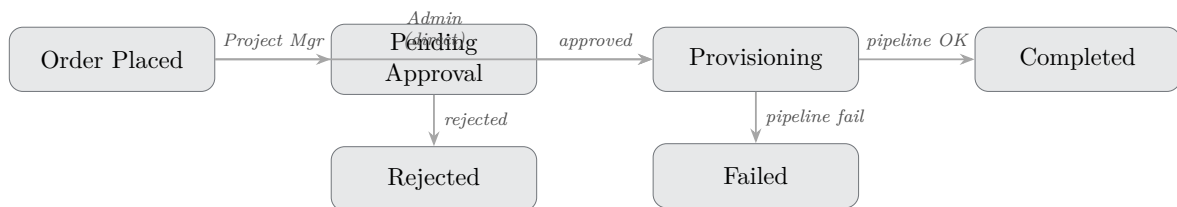


Figure 15.1: Order lifecycle state machine

15.2 Placing an Order

1. Navigate to **Catalog** and select a product.
2. Click **Configure** → select the deployment environment.
3. Select or create a **Project**.
4. Fill in the **Parameters** (required fields are marked).
5. Select or confirm the **Cost Center**.
6. Click **Deploy Now**.

As a **Project Manager** the order status changes to *Pending Approval*. An Admin receives an email notification.

As a **Admin** the GitLab webhook fires immediately.

15.3 Tracking Status

After placing an order, navigate to **My Orders** → **{order}**. Refresh the page to see the latest status. The backend updates the order automatically when the CI provider

sends a pipeline webhook event.

15.4 Using an Existing Deployment as a Template

1. Open **Infrastructure**.
2. Find the resource and click **Use as Template**.
3. The order form opens pre-filled with the original parameters.
4. Modify as needed and submit.

15.5 Decommissioning

1. Open **Infrastructure**.
2. Find the resource and click **Decommission**.
3. Confirm the action.
4. The destroy webhook fires; status changes to *Decommissioning*.
5. On pipeline completion, status becomes *Decommissioned*.

Cascade Decommissioning on Deletion

Deleting a **project**, **product**, or **category** automatically triggers decommissioning for all active infrastructure elements associated with that object before the database record is removed:

- **Project deleted** — all active infrastructure elements of the project receive a destroy webhook call.
- **Product deleted** — all active infrastructure elements provisioned from that product receive a destroy webhook call.
- **Category deleted** — all active infrastructure elements of every product in the category receive a destroy webhook call.

Infrastructure already in status *Decommissioning* or *Decommissioned* is skipped. Dependent database rows (translations, parameters, environment assignments) are removed automatically via PostgreSQL `ON DELETE CASCADE` constraints after the webhooks have been triggered.

Important

Deleting a project, product, or category is irreversible. All associated cloud resources will be destroyed by OpenTofu. Verify there are no active workloads before proceeding.

Chapter 16

Introducing a New Product

This chapter describes the complete process for making a new infrastructure product available in the webshop. Follow the steps sequentially.

16.1 Overview

1. Plan the product
2. Prepare the OpenTofu module
3. Configure the GitLab webhook
4. Create the product in the Admin UI
5. Define parameters (import or manual)
6. Configure deployment environments and pricing
7. Set up AI translations
8. Test the full order flow
9. Publish

16.2 Step 1: Plan the Product

Before touching any UI, answer these questions:

- **What does the product deploy?** (e.g. *Managed PostgreSQL VM on Linode*)
- **Which OpenTofu module delivers it?** (`vm-platform`, `k8s-platform`, or a new module)
- **In which environments is it available?** (e.g. Linode Frankfurt, Linode Amsterdam)
- **What parameters does the user need to provide?** (e.g. `vm_label`, `instance_type`, `db_name`)

- What is the base price in EUR per environment?
- Who assigns the cost center? (project / select / overhead)

16.3 Step 2: Prepare the OpenTofu Module

If the product uses an existing module (`vm-platform` or `k8s-platform`), skip to Step 3. For a new product type:

1. Create a new GitLab repository in the `open-hybrid-cloud` group.
2. Add the standard files: `provider.tf`, `backend.tf`, `variables.tf`, `main.tf`, `outputs.tf`, `.gitlab-ci.yml`.
3. Declare all user-provided parameters as `variable` blocks in `variables.tf` with `description`, `type`, and `default` where applicable:

```
variable "db_name" {
  description = "Name of the PostgreSQL database to create."
  type        = string
}

variable "db_size_gb" {
  description = "Database volume size in GB."
  type        = number
  default     = 20
}
```

4. Test the module locally:

```
tofu init -backend-config="address=..." \
          -backend-config="username=..." \
          -backend-config="password=..."
tofu plan -var="linode_token=$TOKEN" \
          -var="db_name=test" \
          -var="db_size_gb=20"
tofu apply
tofu destroy
```

5. Commit and push to the `main` branch.

16.4 Step 3: Configure the GitLab Webhook

In the GitLab repository:

1. Go to **Settings** → **CI/CD** → **Pipeline trigger tokens**.
2. Click **Add new token** and give it a descriptive name (e.g. *webshop-prod*).
3. Note the **trigger URL** and **token**.

The trigger URL has the form:


```
https://gitlab.example.com/api/v4/projects/<PROJECT_ID>/ref/main/trigger/pipeline
```

16.5 Step 4: Create a Category (if needed)

Under **Administration** → **Categories**:

1. Click **New Category**.
2. Enter a **Name**.
3. Optionally assign a **Category Parameter Set** (see Step 5).
4. Save.

16.6 Step 5: Create the Product

Under **Administration** → **Products** → **New**:

16.6.1 Basic Information

1. Select the **Category**.
2. Enter the product **Name** in the primary language.
3. Enter a detailed **Description** in the primary language.
4. Upload a product **Image** (PNG or JPEG, max 10 MB).
5. Click **Save & Continue**.

16.6.2 Parameters

Choose one of two import methods:

Option A — **Import from `variables.tf`** (recommended):

1. Click **Browse Repository**.
2. Select the **GitLab Source** and the repository.
3. Select the branch (**main**) and navigate to **`variables.tf`**.
4. Click **Import Parameters**.
5. The webshop parses the HCL and creates parameter records for each variable. Review and adjust:
 - **Required** — must the user fill this in?
 - **Sensitive** — should the value be masked in the UI and audit log?
 - **Default value** — pre-fill the order form.

- **Type** — text, number, boolean, or dropdown.

Option B — Manual entry:

Click **Add Parameter** and fill in each field manually.

Note

Parameters whose names match OpenTofu variable names (e.g. `instance_type`) are passed to the webhook as `TF_VAR_instance_type`. The name in the webshop must match the `variable` block name in `variables.tf` exactly.

16.6.3 Deployment Environments

1. Click **Add Environment**.
2. Select an environment from the dropdown (must be pre-configured under Administration → Environments).
3. Enter the **Price** in the base currency.
4. For the webhook, either:
 - Use the environment's default webhook URL, or
 - Enter a product-specific webhook URL and token.
5. Configure the **Cost Center Mode**:
 - Project** Cost center inherited from the project.
 - Select** Orderer selects from the cost center list.
 - Overhead** Always assigns a fixed cost center.
6. Repeat for each environment.

16.6.4 Multiple Webhooks (Pipeline Arrays)

A single product-environment combination can have multiple webhooks executed in order (e.g. first provision the VM, then configure DNS, then register in Vault). Under the webhook section:

1. Click **Add Webhook**.
2. Enter URL, token, and **Execution Order** (1, 2, 3 ...).
3. Webhooks are triggered sequentially in ascending order.

16.7 Step 6: Configure AI Translation

1. Ensure an AI provider is configured (Chapter 7).
2. Open the product in **Administration** → **Products**.

3. Click **Generate AI Translation**.
4. Review the generated translations for all 25 languages.
5. Edit any that require adjustment.
6. Save.

Tip

Generate translations early in the process. The AI translates both the product name and description. Individual translations can be re-generated for specific languages by editing the translation tab directly.

16.8 Step 7: Global and Category Parameter Sets

Global parameters (under **Administration** → **Global Parameters**) apply to every order regardless of product. Use them for organization-wide fields such as a cost center code or project tag that must always be captured.

Category parameters (under **Administration** → **Categories** → **{category}** → **Parameters**) apply to all products within a category. Use them to avoid repeating parameters shared by related products (e.g. all VM products share a **region** parameter).

Parameter hierarchy (applied in this order on the order form):

1. Global parameters
2. Category parameters
3. Product parameters
4. Environment-specific parameters (override per-environment)

16.9 Step 8: Test the Full Order Flow

1. Log in as a **Admin** (Admins trigger webhooks directly).
2. Navigate to **Catalog** and find the new product.
3. Click **Configure**, fill in test parameters, and click **Deploy Now**.
4. Monitor the order under **My Orders** — the status should progress from *Provisioning* to *Completed*.
5. Verify the infrastructure was created in the Linode Cloud Manager.
6. Test the email notification (check the admin email inbox).
7. Test the **Decommission** flow:
 - Open **Infrastructure**.
 - Click **Decommission** on the test resource.

- Verify the destroy pipeline runs and the resource disappears from the Linode Cloud Manager.

Important

Always test decommission in a non-production environment first. A destroy pipeline permanently deletes cloud resources and their data.

16.10 Step 9: Test with a Project Manager Account

1. Log in (or create a test account) as a **Project Manager**.
2. Place an order for the new product.
3. Verify the order shows *Pending Approval*.
4. Switch to the **Admin** account.
5. Approve the order under **Approvals**.
6. Verify the webhook fires and provisioning completes.
7. Verify the Project Manager receives the confirmation email.

16.11 Step 10: Product is Live

Once testing passes:

- The product is immediately visible in the catalog.
- No deployment or restart of the webshop is required.
- Announce the new product to users via the notification service or the shop's subtitle/branding.

Chapter 17

Project Management

17.1 Projects

A *project* is a container for related infrastructure orders. Every order must be associated with a project. Projects have:

- A **Name** and optional **Description**.
- An optional **Cost Center** (used when cost center mode is *Project*).
- An owner (the user who created it).

Project Managers see only their own projects. **Admins** and **Root users** see all projects.

17.2 Infrastructure Overview

The **Infrastructure** page shows all deployed resources grouped by project and environment. Each resource shows:

- Product name and environment.
- Deployment date and current status.
- Key parameters (e.g. VM label, cluster name).
- Buttons: **Use as Template**, **Decommission**.

17.3 Cost Center Assignment

Cost centers are maintained under **Administration** → **Cost Centers**. The three assignment modes give fine-grained control over who can allocate costs:

Project No user input; cost center comes from the project. Use for products where cost is always allocated to the project budget.

Select Orderer picks from the cost center list. Use when individual orders can span cost centers.

Overhead Always assigns a fixed cost center regardless of order. Use for shared services (monitoring, DNS).

Appendix A

Supported Languages

The following 25 languages are supported in the UI and for AI product translation:

Code	Language	Native name
de	German	Deutsch
en	English	English
fr	French	Français
it	Italian	Italiano
es	Spanish	Español
pt	Portuguese	Português
nl	Dutch	Nederlands
pl	Polish	Polski
cs	Czech	Čeština
sk	Slovak	Slovenčina
hu	Hungarian	Magyar
ro	Romanian	Română
bg	Bulgarian	Balgarski (Cyrillic)
hr	Croatian	Hrvatski
sl	Slovenian	Slovenščina
et	Estonian	Eesti
lv	Latvian	Latviešu
lt	Lithuanian	Lietuvių
fi	Finnish	Suomi
sv	Swedish	Svenska
da	Danish	Dansk
el	Greek	Ellinika (Greek script)
mt	Maltese	Malti
ru	Russian	Russkij (Cyrillic)
uk	Ukrainian	Ukrayinska (Cyrillic)

Table A.1: Supported language codes

Appendix B

Environment Variable Reference

Backend (apps/backend/.env)

Variable	Description	Required
<code>DATABASE_URL</code>	PostgreSQL DSN (<code>postgresql://user:pass@host/db</code>)	Yes
<code>JWT_SECRET</code>	HS256 signing secret — min. 32 characters	Yes
<code>ADMIN_EMAIL</code>	Email of the initial Root account	Yes
<code>ADMIN_PASSWORD</code>	Password of the initial Root account	Yes
<code>FRONTEND_URL</code>	Allowed CORS origin (default: <code>http://localhost:3000</code>)	No
<code>SMTP_HOST</code>	SMTP server hostname	No
<code>SMTP_PORT</code>	SMTP port (default: 1025)	No
<code>SMTP_FROM</code>	Sender email address	No
<code>SMTP_USER</code>	SMTP authentication username	No
<code>SMTP_PASS</code>	SMTP authentication password	No
<code>SMTP_TLS</code>	<code>true</code> to enable SMTP TLS	No
<code>EXCHANGE_RATE_API_URL</code>	Exchange rate API endpoint URL	No
<code>ENTRA_TENANT_ID</code>	Microsoft Entra ID tenant ID (future SSO)	No
<code>ENTRA_CLIENT_ID</code>	Entra ID application (client) ID	No
<code>ENTRA_CLIENT_SECRET</code>	Entra ID client secret	No

Table B.1: Backend environment variable reference

Frontend (apps/frontend/.env)

Variable	Description	Required
<code>NEXT_PUBLIC_API_URL</code>	Backend URL reachable from the browser (used for client-side API calls)	Yes
<code>API_URL</code>	Backend URL reachable from the Next.js SSR server	Yes
<code>NEXTAUTH_URL</code>	Canonical public URL of the frontend	Yes
<code>NEXTAUTH_SECRET</code>	NextAuth session cookie encryption secret (min. 32 chars)	Yes

Table B.2: Frontend environment variable reference

Appendix C

OpenTofu Module Variable Reference

C.1 network-core Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
vpc_label	string	VPC label	ohc-vpc-prod
vpc_description	string	VPC description	(see defaults)
subnets	map(obj)	Subnet map (label→cidr)	app + mgmt
firewall_inbound_rules	list(obj)	Inbound firewall rules	SSH + ICMP
dns_domain	string	DNS zone domain (empty=none)	""
tags	list	Resource tags	["infra","network-core"]

Table C.1: network-core variables

C.2 vm-platform Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
vm_label	string	Unique VM label (state name)	—
region	string	Linode region	eu-central
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
firewall_id	number	Base firewall ID	—
instance_type	string	Linode plan	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
volume_size_gb	number	Block storage size (0=none)	0
additional_inbound_rules	list(obj)	Extra firewall rules	[]
dns_zone_id	number	DNS zone ID (null=none)	null
dns_hostname	string	A record hostname	""

Variable	Type	Description	Default
tags	list	Resource tags	["infra","vm"]

Table C.2: vm-platform variables

C.3 k8s-platform Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
cluster_label	string	Unique cluster label	—
region	string	Linode region	eu-central
kubernetes_version	string	K8s version	1.31
vpc_id	number	VPC ID	—
subnet_id	number	Subnet ID	—
node_pools	list(obj)	Node pool configurations	1× g6-standard-2
additional_inbound_rules	list(obj)	Extra firewall rules	[]
dns_zone_id	number	DNS zone ID (null=none)	null
dns_hostname	string	A record hostname	"
tags	list	Resource tags	["infra","k8s"]

Table C.3: k8s-platform variables

C.4 shared-services Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
gitlab_label	string	GitLab CE Linode label	ohc-gitlab
gitlab_instance_type	string	Linode plan for GitLab CE	g6-standard-4
vault_label	string	Vault Linode label	ohc-vault
vault_instance_type	string	Linode plan for Vault	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
allowed_admin_ipv4	list	IPv4 CIDRs for SSH & Vault admin	["0.0.0.0/0"]
tags	list	Resource tags	["infra","shared"]

Table C.4: shared-services variables

C.5 observability Variables

Variable	Type	Description	Default
linode_token	string	Linode API token (sensitive)	—
region	string	Linode region	eu-central
obs_label	string	Linode instance label	ohc-obs
instance_type	string	Linode plan	g6-standard-2
image	string	Boot image	linode/debian12
root_pass	string	Root password (sensitive)	—
ssh_keys	list	Authorized SSH keys	[]
vpc_id	number	VPC ID from network-core	—
subnet_id	number	Subnet ID from network-core	—
volume_size_gb	number	Block storage size in GiB	200
grafana_port	number	Grafana HTTP port	3000
allowed_grafana_ipv4	list	IPv4 CIDRs for Grafana access	["0.0.0.0/0"]
tags	list	Resource tags	["infra","obs"]

Table C.5: observability variables

Appendix D

HTTP Route Reference

All backend REST endpoints are Next.js route handlers under [apps/backend/src/app/api/](#). The frontend is a standard Next.js App Router application; its pages are not listed here.

The **Role** column uses the following abbreviations:

- **public** — no authentication required
- **auth** — any authenticated user (any role)
- **admin** — role `admin` or `root`
- **root** — role `root` only

Method	Path	Description	Role
<i>Authentication & Health</i>			
POST	/api/auth/login	Issue JWT from credentials	public
GET	/api/auth/me	Return caller profile	auth
POST	/api/auth/logout	Stateless no-op (client discards JWT)	auth
GET	/api/health	Health check + bootstrap trigger	public
<i>Catalog</i>			
GET	/api/catalog	List products by category	auth
GET	/api/catalog/{id}	Product detail with environments	auth
GET	/api/catalog/{id}/image	Serve product image (BYTEA)	auth
POST	/api/catalog/{id}/order	Place an order	auth
<i>Orders</i>			
GET	/api/orders	List orders (role-scoped)	auth
GET	/api/orders/{id}	Order detail	auth
POST	/api/orders/{id}/approve	Approve order and trigger pipeline	admin
POST	/api/orders/{id}/reject	Reject order with mandatory note	admin
<i>Projects</i>			
GET	/api/projects	List projects (role-scoped)	auth
POST	/api/projects	Create project	auth

Method	Path	Description	Role
GET	/api/projects/{id}	Project detail	auth
PUT	/api/projects/{id}	Update project	auth
DELETE	/api/projects/{id}	Delete project + cascade decommission	auth
<i>Infrastructure</i>			
GET	/api/infrastructure	Infrastructure overview (role-scoped)	auth
GET	/api/infrastructure/{id}	Infrastructure element detail	auth
POST	/api/infrastructure/{id}/decommission	Trigger destroy pipeline	auth
<i>Admin — Users (role: root)</i>			
GET	/api/admin/users	User list	root
POST	/api/admin/users	Create user	root
PUT	/api/admin/users/{id}	Update user	root
DELETE	/api/admin/users/{id}	Delete user	root
<i>Admin — Catalog (role: root)</i>			
GET	/api/admin/categories	Category list	root
POST	/api/admin/categories	Create category	root
PUT	/api/admin/categories/{id}	Update category	root
DELETE	/api/admin/categories/{id}	Delete category + cascade	root
GET	/api/admin/products	Product list	root
POST	/api/admin/products	Create product	root
GET	/api/admin/products/{id}	Product detail (admin view)	root
PUT	/api/admin/products/{id}	Update product	root
DELETE	/api/admin/products/{id}	Delete product + cascade	root
PUT	/api/admin/products/{id}/image	Upload product image	root
<i>Admin — Environments & CI Sources (role: root)</i>			
GET	/api/admin/environments	Deployment environment list	root
POST	/api/admin/environments	Create environment	root
PUT	/api/admin/environments/{id}	Update environment	root
DELETE	/api/admin/environments/{id}	Delete environment	root
GET	/api/admin/ci-sources	CI source list	root
POST	/api/admin/ci-sources	Create CI source	root
PUT	/api/admin/ci-sources/{id}	Update CI source	root
DELETE	/api/admin/ci-sources/{id}	Delete CI source	root
<i>Admin — CI Repository Browser (role: root)</i>			
GET	/api/admin/ci/projects	Browse CI provider projects	root
GET	/api/admin/ci/branches	Browse branches in a project	root
GET	/api/admin/ci/files	Browse files in a project	root
POST	/api/admin/ci/import-vars	Import vars from <code>variables.tf</code>	root
<i>Admin — Cost Centers (role: root)</i>			
GET	/api/admin/cost-centers	Cost center list	root
POST	/api/admin/cost-centers	Create cost center	root
PUT	/api/admin/cost-centers/{id}	Update cost center	root
DELETE	/api/admin/cost-centers/{id}	Delete cost center	root
<i>Admin — Configuration (role: root)</i>			
GET	/api/admin/branding	Get branding settings	root
PUT	/api/admin/branding	Update branding settings	root
GET	/api/admin/branding/logo	Serve branding logo (BYTEA)	root

Method	Path	Description	Role
PUT	/api/admin/branding/logo	Upload branding logo	root
GET	/api/admin/config	Get app config (SMTP, exchange)	root
PUT	/api/admin/config/smtp	Update SMTP config	root
PUT	/api/admin/config/exchange	Update exchange rate API config	root
<i>Admin — Audit (role: admin)</i>			
GET	/api/admin/audit	Paginated audit log	admin
<i>CI Provider Webhooks (inbound, token-authenticated)</i>			
POST	/api/webhooks/gitlab/pipeline	GitLab pipeline status event	public
POST	/api/webhooks/github/pipeline	GitHub Actions workflow event	public
POST	/api/webhooks/bitbucket/pipeline	Bitbucket pipeline status event	public

Table D.1: Complete REST API route reference

Appendix E

Requirements Traceability Matrix

The following table maps each functional and non-functional requirement to the implementation location and the handbook section that describes it.

ID	Requirement	Implementation	Section
FA-01	User authentication (local credentials + JWT)	<code>apps/backend/src/app/api/auth/login/route.ts</code> , <code>apps/backend/src/lib/auth/</code>	§8
FA-02	Role-based access control (admin, root, project_manager)	<code>apps/backend/src/lib/auth/middleware.ts</code>	§8
FA-03	Product catalog with categories	<code>apps/backend/src/app/api/catalog/</code>	§15
FA-04	Configurable product parameters	<code>apps/backend/src/lib/db/schema.ts</code> (parameters table)	§16.6
FA-05	Order placement flow	<code>apps/backend/src/app/api/catalog/[id]/order/route.ts</code>	§15
FA-06	Order approval workflow	<code>apps/backend/src/app/api/orders/[id]/approve/route.ts</code>	§15
FA-07	CI provider webhook trigger (GitLab / GitHub / Bitbucket)	<code>apps/backend/src/lib/ci/index.ts</code>	§4.5
FA-08	Inbound pipeline status via CI webhooks (no polling)	<code>apps/backend/src/app/api/webhooks/[provider]/pipeline/[id]/route.ts</code> , <code>apps/backend/src/lib/webhook/handler.ts</code>	§4.5
FA-09	Infrastructure overview	<code>apps/backend/src/app/api/infrastructure/</code>	§17.2
FA-10	Decommission (destroy) flow	<code>apps/backend/src/app/api/infrastructure/[id]/decommission/route.ts</code>	§15.5
FA-09.5–9.8	Cascade decommission on project/product/category delete	<code>apps/backend/src/app/api/projects/[id]/route.ts</code>	§15.5
FA-11	Audit log	<code>apps/backend/src/app/api/admin/audit/route.ts</code>	§9
FA-13	Multi-currency support	<code>apps/backend/src/lib/db/schema.ts</code> (exchange_rates), <code>apps/backend/src/app/api/admin/config/exchange/</code>	§7.6
FA-14	Branding configuration	<code>apps/backend/src/app/api/admin/branding/</code>	§7.5
NFA-01.1	Docker deployment (two containers)	<code>apps/backend/Dockerfile</code> , <code>apps/frontend/Dockerfile</code> , <code>infra/docker-compose.yml</code>	§6.3
NFA-01.2	Kubernetes deployment	<code>infra/helm/open-hybrid-cloud/</code>	§6.4
NFA-01.3	Schema management via Drizzle ORM	<code>apps/backend/src/lib/db/schema.ts</code> , <code>make db-push</code>	§3
NFA-02.1	HTTPS enforcement	Nginx config (reverse proxy, TLS termination)	§6.3
NFA-02.2	JWT authentication (HS256, 24 h)	<code>apps/backend/src/lib/auth/jwt.ts</code> , jose library	§3
NFA-02.3	bcrypt password hashing	<code>apps/backend/src/app/api/auth/login/route.ts</code> , <code>bcryptjs</code>	§3
NFA-02.4	Role-enforced API routes	<code>apps/backend/src/lib/auth/middleware.ts</code>	§8
NFA-04	Audit trail for all state changes	<code>apps/backend/src/lib/db/schema.ts</code> (audit_log)	§9

ID	Requirement	Implementation	Section
----	-------------	----------------	---------

Table E.1: Requirements traceability matrix

Appendix F

Compiling This Handbook

The handbook source is at [docs/handbook.tex](#). Compile with:

```
# From the repo root
make docs

# Or manually (run twice for ToC/references)
cd docs
pdflatex handbook.tex
pdflatex handbook.tex
```

Required LaTeX packages: lmodern, geometry, fancyhdr, xcolor, tikz, pgf, booktabs, longtable, tabularx, listings, tcolorbox, hyperref, enumitem, float, caption, graphicx, microtype, parskip, pdflscape, pifont.

All packages are available in TeX Live 2023+ and MiKTeX 23+.